Atividade Prática

- **Tema:** autocomplete.
- **Prazo de entrega:** 15 de Junho.

Envie o notebook **executado** em formato **ipynb** pelo formulário.

Enunciado:

Vamos realizar uma tarefa de ***autocomplete*** com o conjundo de dados IMDB movie reviews, também disponível no diretório $\mathrm{dataset}/$. Nesta tarefa, iremos encontrar o próximo *token* mais provável. Para isso, realize os seguintes passos atentamente:

1º Passo:

- Importe o *dataset* e realize uma amostragem dos dados a fim de capturar 500 textos de revisão;
- Defina os dados textuais em uma lista.

Resolução item 1

Vamos adicionar na resolução do item 1 algumas atividades que nos ajudarão a executar as tarefas seguintes. Inicialmente saltamos o import do google drive pois estamos trabalhando com dados e processamento local.

In [16…

```python
# não é necessario montar o google drive por que estamos trabalhando com dados e processamento local
#from google.colab import drive
#drive.mount('/content/drive/')
```

Imports e definições globais

Definimos imports e algumas variáveis globais utilizadas ao longo do notebook.

In [26…

```python
from collections import Counter
from keras import layers, Model
import keras_tuner as kt
from keras.metrics import SparseTopKCategoricalAccuracy
from keras.optimizers import Adam
from keras.callbacks import EarlyStopping, ReduceLROnPlateau, CSVLogger
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import tensorflow as tf
from matplotlib.ticker import FuncFormatter
import os
from pathlib import Path
from sklearn.metrics import mean_absolute_error, mean_squared_error, median_absolute_error, r2_score
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import MinMaxScaler
import tensorboard
from tensorboard.backend.event_processing.event_accumulator import EventAccumulator

from sklearn.metrics import ConfusionMatrixDisplay, accuracy_score, classification_report, confusion_matrix, f1_score, precision_score, recall_score, roc_auc_score
from tensorflow.keras.preprocessing.sequence import pad_sequences

RANDOM_STATE = 42

N_SAMPLES_TRAIN = 3_000 # numero de amostras de treinamento
N_SAMPLES_TEST = 500 # numero de amostras de teste

MAX_LEN = 16 # tamanho máximo das sequencias de tokens
```

Verificamos se a GPU está disponível.

In [16…

```python
print(tf.config.list_physical_devices())
print(tf.config.list_physical_devices("GPU"))
#tf.debugging.set_log_device_placement(True)
```

```
[PhysicalDevice(name='/physical_device:CPU:0', device_type='CPU'), PhysicalDevice(name='/physical_device:GPU:0', device_type='GPU')]
[PhysicalDevice(name='/physical_device:GPU:0', device_type='GPU')]
```

Leitura dos dados

Na leitura dos dados vamos utilizar as variáveis globais `N_SAMPLES_TRAIN` e `N_SAMPLES_TEST` para leer um total de `N_SAMPLES_TRAIN + N_SAMPLES_TEST` amostras, sendo `N_SAMPLES_TRAIN` as amostras exclusivas de treino/validação e `N_SAMPLES_TEST` as amostras exclusivas para teste do modelo.

No momento da leitura do dataframe original aplicamos randomização na ordem dos elementos com `random_state=RANDOM_STATE` para garantir reprodutibilidade do experimento. Em seguida selecionamos em df_test as amostras desejadas, lembrando que já foram randomizadas na leitura, e sobrescrevemos o dataframe original com o conjunto de treino/validação. Mostramos finalmente a quantidade de amostras de cada conjunto e alguns exemplos de dados selecionados para conferência.

In [16…

```python
df = (
    pd.read_csv("./dataset/imdb.csv")
    .sample(
        frac=1,
        random_state=RANDOM_STATE
    )
    .reset_index(drop=True)
)

df_test = (
    df
    .iloc[
        N_SAMPLES_TRAIN:
        N_SAMPLES_TRAIN + N_SAMPLES_TEST
    ]
    .copy()
)

df = (
    df
    .iloc[:N_SAMPLES_TRAIN]
    .copy()
)

print("Desenvolvimento:", df.shape)
print("Teste externo:  ", df_test.shape)

df.head()
```

```
Desenvolvimento: (3000, 2)
Teste externo:   (500, 2)
```

Out[16…

	reviews	label
0	River's Edge is more than just the story of a ...	1
1	Evening is the beautiful story of the flawed l...	1
2	I question anyone saying they don't care for t...	1
3	This docu-drama is what you would expect from ...	1
4	As several posters have "hinted," this is a so...	0

Pré processamento dos dados

Inicialmente adotamos uma estratégia mais tradicional em relação aos exercícios que vinhamos trabalhando até então. Entretanto, durante as modelagem da rede, percebemos que o pré processamento especifico implementado abaixo ajudaria a melhorar nosso modelo. Em particular, notamos que o modelo poderia se beneficiar se aplicassemos algumas expansões de contrações em inglês específicas. Como durante o processo de modelagem adicionamos o GloVe com embeddings pré-treinados, notamos uma quantidade importante de tokens que não estavam nos embeddings do GloVe. Investigando quais tokens eram esses, notamos que uma grande parte representavam expansões que não estavam sendo bem tratadas no pre-processamento. Por isso, implementamos as funções a seguir.

In [16…

```python
CONTRACOES_COMUNS = [
    # Casos específicos com "'s" interpretado como "is".
    (r"\bit's\b", "it is"),
    (r"\bthat's\b", "that is"),
    (r"\bthere's\b", "there is"),
    (r"\bwhat's\b", "what is"),
    (r"\bwho's\b", "who is"),
    (r"\bhere's\b", "here is"),
    (r"\bwhere's\b", "where is"),
    (r"\bhow's\b", "how is"),

    # Contrações irregulares.
    (r"\bwon't\b", "will not"),
    (r"\bcan't\b", "can not"),
    (r"\bain't\b", "is not"),

    # Contrações regulares.
    (r"n't\b", " not"),
    (r"'re\b", " are"),
    (r"'ve\b", " have"),
    (r"'ll\b", " will"),
    (r"'m\b", " am"),

    # Simplificação: "'d" pode significar "would" ou "had".
    (r"'d\b", " would"),

    # Possessivos remanescentes.
    (r"'s\b", ""),

    # Apóstrofos finais, como em "James'" ou "actors'".
    (r"'\b", ""),
]

def expandir_constracoes_comuns(x):
    """
    Expande contrações comuns da língua inglesa.

    As regras são aplicadas sequencialmente. A ordem é importante:
    substituições específicas devem ocorrer antes das regras genéricas.
    """
    for padrao, substituicao in CONTRACOES_COMUNS:
        x = tf.strings.regex_replace(
            x,
            padrao,
            substituicao
        )

    return x
```

E em seguida utilizamos a função de expansão de contrações junto com a função de pré-processamento definida a seguir.

In [16…

```python
def Preprocessor(x):
    """
    Normaliza os textos antes da vetorização.
    """

    # Converte tudo para minúsculo.
    x = tf.strings.lower(x)

    # Normaliza diferentes formatos de apóstrofo.
    x = tf.strings.regex_replace(
        x,
        r"[‘’`´]",
        "'"
    )

    # Remove tags HTML.
    x = tf.strings.regex_replace(
        x,
        r"<[^>]+>",
        " "
    )

    # Remove links HTTP e HTTPS.
    x = tf.strings.regex_replace(
        x,
        r"https?://\S+",
        " "
    )

    # Substitui quebras de linha literais e reais por espaços.
    x = tf.strings.regex_replace(
        x,
        r"\\n|\n",
        " "
    )

    # Expande contrações antes de remover caracteres especiais.
    x = expandir_constracoes_comuns(x)

    # Substitui caracteres especiais por espaços.
    # Evita concatenar palavras:
    # "low-budget"      -> "low budget"
    # "writer/director" -> "writer director"
    x = tf.strings.regex_replace(
        x,
        r"[^\w\s]",
        " "
    )

    # Remove espaços duplicados.
    x = tf.strings.regex_replace(
        x,
        r"\s+",
        " "
    )

    # Remove espaços extras no início e no final.
    x = tf.strings.strip(x)

    return x
```

Por fim, deixamos registrados alguns casos de contrações e algumas frases que nos auxiliaram a determinar o método de pré processamento para ajudar na revisão da função.

In [16…

```python
preprocessor_text_texts = [
    "   Hello,   the  world's our !   ",
    "you´ve         ",
    "weren`t        ",
    "clich's        ",
    "they‘’ve        ",
    "hadn’t         ",
    "could’ve       ",
    "bmovie        ",
    "writer-director",
    "shouldn't      ",
    "low-budget     ",
    "interferencia ",
    "would've       ",
    "clich'd        ",
    "they'd         ",
    "sam-pedros     ",
    "over-the-top    ",
    "astrix        ",
    "reallife      ",
    "pg-13          ",
    "must-see       ",
    "le`lands       ",
    "it'll          ",
    "It's a movie that I didn't like.",
    "You've probably seen Hitchcock's films.",
    "The low-budget writer/director couldn't finish it.",
    "They'll say it wasn't Chaplin's best work.",
    "James' film isn't one of the actors' best performances."
]

for text in preprocessor_text_texts:
    print(f"[{text}] => [{Preprocessor(text).numpy().decode("utf-8")}]")
```

```
[   Hello,   the  world's our !   ] => [hello the world our]
[you´ve         ] => [you have]
[weren`t        ] => [were not]
[clich's        ] => [clich]
[they‘’ve        ] => [they have]
[hadn’t         ] => [had not]
[could’ve       ] => [could have]
[bmovie        ] => [bmovie]
[writer-director] => [writer director]
[shouldn't      ] => [should not]
[low-budget     ] => [low budget]
[interferencia ] => [interferencia]
[would've       ] => [would have]
[clich'd        ] => [clich would]
[they'd         ] => [they would]
[sam-pedros     ] => [sam pedros]
[over-the-top    ] => [over the top]
[astrix        ] => [astrix]
[reallife      ] => [reallife]
[pg-13          ] => [pg 13]
[must-see       ] => [must see]
[le`lands       ] => [lelands]
[it'll          ] => [it will]
[It's a movie that I didn't like.] => [it is a movie that i did not like]
[You've probably seen Hitchcock's films.] => [you have probably seen hitchcock films]
[The low-budget writer/director couldn't finish it.] => [the low budget writer director could not finish it]
[They'll say it wasn't Chaplin's best work.] => [they will say it was not chaplin best work]
[James' film isn't one of the actors' best performances.] => [james film is not one of the actors best performances]
```

A seguir apresentamos alguns exemplos das reviews depois de aplicar a função de pré-processamento.

In [17…

```python
df_reviews = df[["reviews"]].copy()
df_reviews["reviews"] = df_reviews["reviews"].map(
    lambda x: Preprocessor(x).numpy().decode("utf-8")
)
df_reviews.head(5).style.set_properties(
    subset=["reviews"],
    **{"white-space": "pre-wrap", "max-width": "800px", "text-align": "left"}
)
```

Out[17…

	reviews
0	river edge is more than just the story of a murder it is an indictment of the wave of apathy that has plagued pockets of youth for decades now our main characters are a group of what my high school would have referred to as stoners one of them just decided to strangle his girlfriend because she apparently had the nerve to talk sh t to him this dangerous young man played by daniel roebuck has an intensity that will startle you he takes the other kids in the group out to show them his girlfriend body and strangely enough nobody seems really freaked out about it the balance of the film punctuates the desperate circumstances in which these people live and how guilt is eventually able to worm its way inside even these apathetic kids river edge is certainly not condoning or championing the behavior of its characters we clearly see the dangers presented by such unchecked apathy and having only the desire to get drunk or high we are shown the dysfunctional home lives these kids have and perhaps this is meant to explain their awful behavior but could it only be unstable homes that lead to this type of destructive living crispin glover is his usual whacked out self he drives around town in a state of complete paranoia after the murder he tries to sympathize with why his friend had to commit the crime but clearly he does not understand what could have made the young man do it dennis hopper plays an older man these kids get their dope from though he is certainly a rebellious figure he cannot relate to the apathy and rage of the younger generation witness his confused reaction as roebuck describes how killing his girlfriend made him feel as wild as hopper character is even he knows this young man should not be walking the face of the earth what will you think of these kids well there is really no way to like their characters some are less despicable than others but you cannot help either hating or feeling sorry for them twenty years later there is still an apathy alive and well in a great many young people out there when do these kids get lost what makes so many of them want to act anti social this film and the questions it poses about teenagers will stick with you for a while that is a fact 8 of 10 stars the hound
1	evening is the beautiful story of the flawed love of a mother the movie split in time is magically shot amazingly acted and has a touching script vanessa redgrave plays anne grant lord a woman sun setting out of life lying in her bed her mind remembering and misfiring she recalls her first mistake claire danes plays the young anne giving a youthful vitality to dying bed ridden woman daughters nina toni collette and constance natasha richardson try to decipher the real story from the disheartening dementia her first mistake revolves around harris arden patrick wilson the man her best friend lila mamie gummer deeply loved the daughters must come to terms with their mother past and their futures the cast is glowing in evening the collective acting energy of this movie could have powered the equipment for the production of this entire film i am so glad to see claire danes working again especially in this role she is so young and alive fully living the joys mistakes and heartbreak of young anne first mistake this is a true feat when you realize she is playing a woman dying in bed when her life overwhelms her you can feel her desire to crack and her hopeless hope that she will not some of her facial expressions grinded on me a little but over all her performance was so radiant i was left with that only as a side note toni collette continues to prove that you can be a powerful actress without being a super model she plays the black sheep of the family a little lost nina finds a great deal of strength in her mother mistake collette delicately avoids creating a cruel character who revels in the mistakes of her mother instead choosing the wiser path of learning from her mother mistakes there is a great deal of infighting between nina and her sister constance their fights remind me of ones i have with my sister all the time mamie gummer who plays anne youthful best friend is wonderful her character is stuck between her heart and her status in society even when she is crying and her heart is breaking she is incredibly regal and charming i can not wait to see her act in something else in the future vanessa redgrave performance is very hard for me to describe her talent at making her mental status ambiguous without being wacko or even especially tragic is why it is so powerful the audience does not know if she is making up the story because she is slipping away or if these events truly happened physically and emotionally speaking redgrave is acting in a box not much physical space and limited emotional range might have been a stunner to a lesser actress but she makes the limitations work for her i was constantly amazed the movie is definitely woman focused but the men in the movie are not just accessories patrick wilson is mesmerizing as harris it is no wonder that everyone in the movie is in love with him i sure was buddy wittenborn is lila brother spiraling out of control hugh dancy spirals buddy out of control without sending his acting down the drain glen close has my favorite scene in the movie it reminded me of the famous scene from monster ball it is terrible and jaw dropping grief i was utterly stunned the one acting disappointment was natasha richardson while her fight scenes were memorable most of her acting reeks of melodrama it would have suited her to take an acting bath before we had to breathe her stink it is a good thing she was not in charge of the visuals the visuals of the movie are sparkling cinematographer gyula pados could not make a film richer in color light so perfectly matched to mood and emotion the visual concepts of the flash back sequences are powerful and resonating there were many scenes that could have been stopped printed mounted and sold as art i admit it i cried evening is a powerful movie evening is defiantly a chick flick but a really great chick flick if you want to impress a woman with a movie choice pick evening
2	i question anyone saying they do not care for this movie some reviewers have said it did not have enough action some said it was too long etc do not listen to them if you like shawshank redemption and or braveheart you will definitely love this movie the acting performances are superb tim roth john hurt and jessica lang are allsuperb and liam neeson does an admarible job and is a very imposing character because of his size the cinemaphotography was brilliant and breathtaking it is onw of the few movies i have seen in my life along with shawshank that was virtually flawless from casting directing writing acting etc i was amazed this was not in the top 50 or 100 movies reviewed i felt so passionately about it that i just registered with imdb so i could let everyone know the real scoop i have seen this movie about 10 times each time with a different person and everyone has loved it you must be awake and pay attention carefully for the first 30 minutes because they introduce quite a few characters in the beginning if you have the attention span longer than most of these juvenille kids writing reviews for the movies on this site than you will love this movie come on all 3 lord of the rings movies in the top 9 and the matrix at 32 that should show you the age range of most reviewers here this is a top 50 movie
3	this docu drama is what you would expect from richard attenborough the man who gave us gandhi beautifully photographed compellingly casted well written in the measured literate manner that hollywood discarded in the 30 and scrupulously accurate it stands out as a genre film excelling in its portrayal of native american or more appropriately for its canadian setting first nations culture and standing with black robe as a wonderfully photographed piece of canoe country and its culture here circa 1934 this idyllic portrait derives drama from its subject archie grey owl belaney a scot raised in hastings england by maiden aunts who became so obsessed with the red indian tales of his childhood that he went to canada disappeared into the woods and became a trapper and adopted son of an ojibway band he was a vain man with a habit of marrying and abandoning indian brides none of whom seem to have thought less of him for it for he was also an extraordinarily charming and picaresque character one of his wives one smarter than he by most accounts propelled him into fame as a writer and early advocate for protecting the wild country of the north and this forms the focus of attenborough tale the chemistry between brosnan and annie galipeau as grey owl wife pony is engaging and if not firey is nonetheless quite touching a good film when you need some time from the madding crowd
4	as several posters have hinted this is a sorry star wars ripoff now if you are going to rip off star wars at least do it right battlestar galactica did and there were a few other space operas that did not do a bad job of it but this is definitely not one of them david mendenhall the juvenile lead actually is not too bad though he goes overboard on the cute factor every so often vince edwards has not improved much as an actor since his ben casey days if anything he even more wooden than he was ten the other performances are nothing to write home about either even worse are the special effects the best you can say about them is that they are lousy it is glaringly obvious that the aliens are simply actors wearing rubber masks with a little foam or latex slopped on them and the battle scenes between edwards raiders and the aliens are poorly staged and badly shot a very weak effort from roger corman skip it

Verificamos quantas amostras foram carregadas e se alguma amostra ficou vazia em função do pré-processamento.

In [17…

```python
print(f"Quantidade de amostras: {len(df)}")

print(f"\nValores ausentes (NaN): {df[["reviews"]].isna().sum()}")
print()
```

```
Quantidade de amostras: 3000

Valores ausentes (NaN): reviews    0
dtype: int64
```

Nenhuma amostra ficou vazia depois do pré-processamento. Então, apresentamos os 20 tokens mais frequentes na nossa amostra e verificamos se existe algum token estranho ou indesejado que ocorra com alta frequência.

In [17…

```python
tokens = " ".join(df_reviews["reviews"]).split()
token_counts = Counter(tokens)

token_counts.most_common(20)
```

Out[17…

```
[('the', 41168),
 ('and', 20057),
 ('a', 19854),
 ('of', 17745),
 ('to', 16565),
 ('is', 16183),
 ('it', 11632),
 ('in', 11494),
 ('i', 10615),
 ('this', 8964),
 ('that', 8785),
 ('not', 7745),
 ('was', 6249),
 ('as', 5540),
 ('with', 5412),
 ('for', 5244),
 ('but', 5173),
 ('movie', 5128),
 ('film', 4827),
 ('you', 4264)]
```

Definindo `VOCAB_SIZE`

O próximo passo é gerar insumo necessário para escolher adequadamente o tamanho do nosso vocabulário. Para tanto, adicionamos uma coluna extra com a quantidade de tokens de cada amostra e fazemos uma breve descrição estatística da distribuição dessa amostra. Mais adiante vamos gerar um gráfico mais rico sobre essa distribuição.

In [17…

```python
df_reviews["length"] = df_reviews["reviews"].str.split().str.len()
df_reviews["length"].describe()
```

Out[17…

```
count    3000.000000
mean      239.111000
std       183.397587
min        25.000000
25%       130.000000
50%       174.000000
75%       289.000000
max      2483.000000
Name: length, dtype: float64
```

Definimos uma função para gerar o gráfico de distribuição da quantidade de tokens.

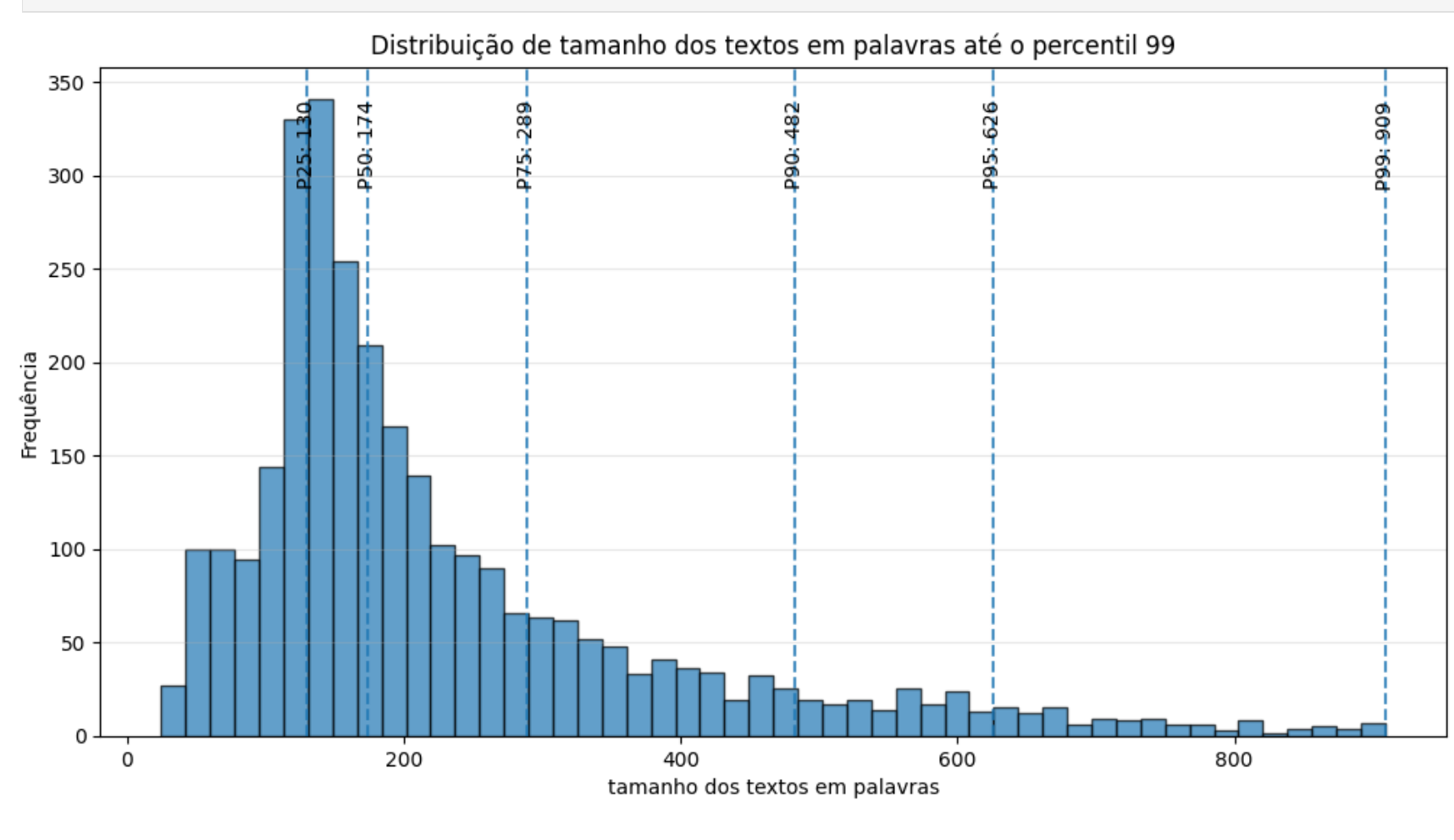
In [17…

```python
def br_number(value, decimals=0):
    '''
    Formata um número para o formato brasileiro com a quantidade de casas decimais desejada.
    '''
    formatted = f"{value:,.{decimals}f}"
    return formatted.replace(",", "X").replace(".", ",").replace("X", ".")

def formatar_numero_br(x, casas_decimais=1):
    '''
    Formata um número utilizando vírgula como separador decimal
    e ponto como separador de milhar.
    '''
    return (
        f"{x:,.{casas_decimais}f}"
        .replace(",", "_")
        .replace(".", ",")
        .replace("_", ".")
    )


def plotar_histograma_com_percentis(
    y,
    nome="variável",
    limite_percentil=0.99,
    percentis_marcados=[0.25, 0.50, 0.75, 0.90, 0.95, 0.99],
    bins=50,
    usar_milhoes=True,
    casas_decimais=1
):
    '''
    Plota um histograma dos dados, juntamente com os percentis que desejamos
    marcar no gráfico.
    '''
    limite = y.quantile(limite_percentil)
    y_filtrado = y[y <= limite]

    plt.figure(figsize=(12, 6))
    plt.hist(y_filtrado, bins=bins, edgecolor="black", alpha=0.7)

    if usar_milhoes:
        plt.gca().xaxis.set_major_formatter(
            FuncFormatter(
                lambda x, pos: formatar_numero_br(
                    x / 1_000_000,
                    casas_decimais
                )
            )
        )
        xlabel = f"{nome} em milhões"
    else:
        plt.gca().xaxis.set_major_formatter(
            FuncFormatter(
                lambda x, pos: formatar_numero_br(
                    x,
                    casas_decimais
                )
            )
        )
        xlabel = nome

    altura_max = plt.ylim()[1]

    for p in percentis_marcados:
        valor = y.quantile(p)

        if valor <= limite:
            if usar_milhoes:
                valor_formatado = (
                    f"{formatar_numero_br(valor / 1_000_000, casas_decimais)} mi"
                )
            else:
                valor_formatado = formatar_numero_br(valor, casas_decimais)

            plt.axvline(valor, linestyle="--", linewidth=1.5, alpha=0.8)
            plt.text(
                valor,
                altura_max * 0.95,
                f"P{int(p * 100)}: {valor_formatado}",
                rotation=90,
                ha="center",
                va="top"
            )

    plt.title(
        f"Distribuição de {nome} até o percentil "
        f"{int(limite_percentil * 100)}"
    )
    plt.xlabel(xlabel)
    plt.ylabel("Frequência")
    plt.grid(True, axis="y", alpha=0.3)
    plt.show()
```

E exibimos o gráfico de distribuição de quantidade de tokens com os percentis 25, 50, 75, 90, 96 e 99.

In [17…

```python
plotar_histograma_com_percentis(
    df_reviews["length"],
    nome="tamanho dos textos em palavras",
    limite_percentil=0.99,
    percentis_marcados=[0.25, 0.50, 0.75, 0.90, 0.95, 0.99],
    bins=50,
    usar_milhoes=False,
    casas_decimais=0
)
```



A seguir, verificamos a quantidade de palavras de nossas amostras e a quantidade de tokens (palavras distintas) da amostra completa.

In [17…

```python
token_counts = Counter(
    token
    for comment in df_reviews["reviews"]
    for token in comment.split()
)
# Ordena os tokens do mais frequente para o menos frequente
frequencias_ordenadas = sorted(token_counts.values(), reverse=True)

total_ocorrencias = sum(frequencias_ordenadas)
total_tokens_unicos = len(frequencias_ordenadas)

print(f"Total de ocorrências de palavras: {br_number(total_ocorrencias)}")
print(f"Quantidade de palavras distintas: {br_number(total_tokens_unicos)}")
```

```
Total de ocorrências de palavras: 717.333
Quantidade de palavras distintas: 31.426
```

Vamos gerar um gráfico com a cobertura de tokens para um determinado `VOCAB_SIZE` para auxiliar na escolha do melhor valor. Para tanto vamos calcular a cobertura para cada uma das opções a seguir.

In [17…

```python
vocab_sizes = [
     2_500,
     5_000,
     7_500,
    10_000,
    12_500,
    15_000,
    20_000,
    25_000,
    30_000,
    32_500
]
```

In [17…

```python
resultados_vocabulario = []

for vocab_size in vocab_sizes:
    tamanho_efetivo = min(vocab_size, total_tokens_unicos)

    ocorrencias_cobertas = sum(frequencias_ordenadas[:tamanho_efetivo])
    ocorrencias_oov = total_ocorrencias - ocorrencias_cobertas

    resultados_vocabulario.append({
        "VOCAB_SIZE": vocab_size,
        "palavras incluídas": tamanho_efetivo,
        "ocorrências cobertas": ocorrencias_cobertas,
        "cobertura (%)": ocorrencias_cobertas / total_ocorrencias * 100,
        "ocorrências OOV": ocorrencias_oov,
        "OOV (%)": ocorrencias_oov / total_ocorrencias * 100
    })

df_vocabulario = pd.DataFrame(resultados_vocabulario)

df_vocabulario.style.format({
    "VOCAB_SIZE": "{:,.0f}",
    "palavras incluídas": "{:,.0f}",
    "ocorrências cobertas": "{:,.0f}",
    "cobertura (%)": "{:.2f}",
    "ocorrências OOV": "{:,.0f}",
    "OOV (%)": "{:.2f}"
})
```

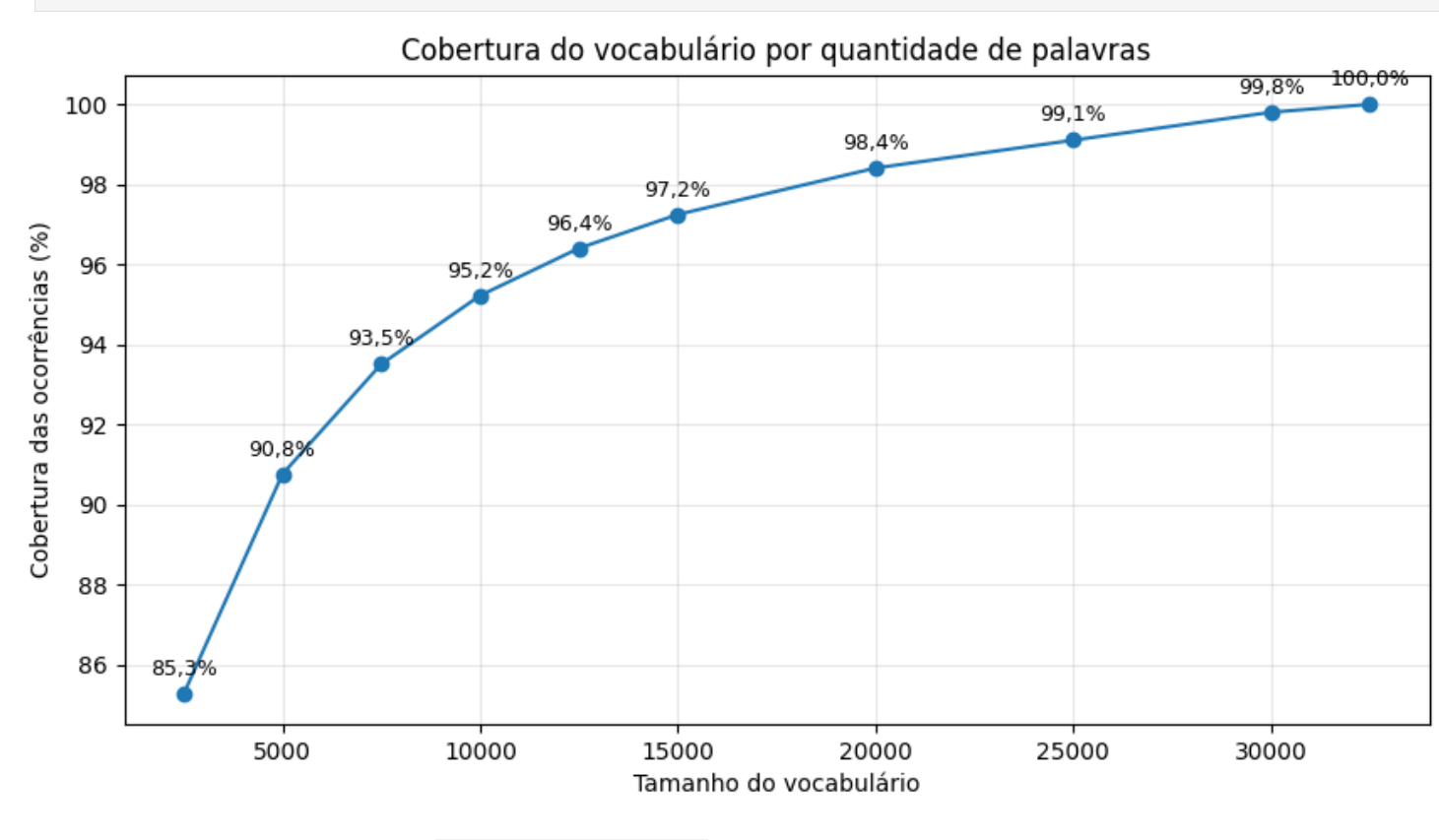
Out[17…

	VOCAB_SIZE	palavras incluídas	ocorrências cobertas	cobertura (%)	ocorrências OOV	OOV (%)
0	2,500	2,500	611,584	85.26	105,749	14.74
1	5,000	5,000	651,045	90.76	66,288	9.24
2	7,500	7,500	670,765	93.51	46,568	6.49
3	10,000	10,000	683,058	95.22	34,275	4.78
4	12,500	12,500	691,536	96.40	25,797	3.60
5	15,000	15,000	697,584	97.25	19,749	2.75
6	20,000	20,000	705,907	98.41	11,426	1.59
7	25,000	25,000	710,907	99.10	6,426	0.90
8	30,000	30,000	715,907	99.80	1,426	0.20
9	32,500	31,426	717,333	100.00	0	0.00

Por fim criamos um gráfico de VOCAB_SIZE X Cobertura de tokens do vocabulário.

In [17…

```python
def plotar_cobertura_vocabulario():
    plt.figure(figsize=(10, 5))

    plt.plot(
        df_vocabulario["VOCAB_SIZE"],
        df_vocabulario["cobertura (%)"],
        marker="o"
    )

    for x, y in zip(
        df_vocabulario["VOCAB_SIZE"],
        df_vocabulario["cobertura (%)"]
    ):
        plt.annotate(
            f"{y:.1f}%".replace(".", ","),
            xy=(x, y),
            xytext=(0, 8),
            textcoords="offset points",
            ha="center",
            fontsize=9
        )

    plt.title("Cobertura do vocabulário por quantidade de palavras")
    plt.xlabel("Tamanho do vocabulário")
    plt.ylabel("Cobertura das ocorrências (%)")
    plt.grid(True, alpha=0.3)

    plt.show()

plotar_cobertura_vocabulario()
```



Pelo gráfico podemos observar que `VOCAB_SIZE = 5_000` nos permite alcançar cobertura de 90% das palavras da nossa amostra. Os demais tokens serão identificados todos como `[UNK]`, o token pré-definido para representar palavras desconhecidas.

In [18…

```python
VOCAB_SIZE = 5_000
```

Dividindo os conjuntos de treino, validação e testes e levantando informações para preparação das amostras de sequencias

Nos passos seguintes, vamos gerar, a partir de cada review, sequências de tokens com tamanho máximo igual a 16. Cada sequência será utilizada como entrada para que o modelo aprenda a prever o token seguinte.

Entretanto, antes de gerar essas sequências, é necessário separar os conjuntos de treinamento e teste. Essa separação deve ser realizada no nível das reviews completas, e não no nível das sequências geradas. Caso contrário, trechos muito semelhantes de uma mesma review podem aparecer simultaneamente nos dois conjuntos. Nessa situação, o modelo seria avaliado com sequências muito próximas daquelas utilizadas durante o treinamento, produzindo métricas excessivamente otimistas.

O vocabulário também deve ser construído utilizando apenas as reviews do conjunto de treinamento. Dessa forma, informações específicas dos textos reservados para teste não influenciam a preparação do modelo. Palavras que aparecem apenas no conjunto de teste serão representadas pelo token `[UNK]`.

Como as reviews possuem comprimentos variados, cada uma delas gera uma quantidade diferente de sequências. Portanto, uma separação contendo 80% das reviews para treinamento e 20% para teste não necessariamente produzirá a mesma proporção entre as sequências geradas. Após a divisão, é necessário verificar a quantidade de amostras resultante em cada conjunto e avaliar se a proporção permanece adequada. Dado que queremos janelas deslizantes de *até* 16 tokens, admitindo janelas com menos tokens (preenchidos por padding), podemos facilmente calcular o numero de sequencias geradas a partir de cada documento como sendo: `sequence_count = length - 1`. Vamos então adicionar essa coluna no `df_reviews`

Por fim vamos verificar se, após a separação dos conjuntos de treino e validação, a proporção de sequencias a serem geradas permanece aceitável. Certamente a distribuição se modificará, mas esperamos que ela se mantenha com uma proporção aceitável para o objetivo de treino e validação do modelo.

In [18…

```python
df_reviews["sequence_count"] = (df_reviews["length"] - 1).clip(lower=0)

df_reviews["sequence_count"].describe()
```

Out[18…

```
count    3000.000000
mean      238.111000
std       183.397587
min        24.000000
25%       129.000000
50%       173.000000
75%       288.000000
max      2482.000000
Name: sequence_count, dtype: float64
```

In [18…

```python
df_test = df_test[["reviews"]].copy() # copiamos somente a coluna reviews do dataframe original

df_test["reviews"] = ( # aplicamos o pré-processamento nas reviews do conjunto de teste
    Preprocessor(
        tf.constant(
            df_test["reviews"].tolist()
        )
    )
    .numpy()
    .astype(str)
)

df_test["length"] = ( # calculamos o número de tokens de cada review
    df_test["reviews"]
    .str.split()
    .str.len()
)

df_test["sequence_count"] = ( # calculamos o número de sequências de cada review
    df_test["length"] - 1
).clip(lower=0)

df_train, df_val = train_test_split( # separamos o conjunto de treino e validação
    df_reviews,
    test_size=0.20,
    random_state=RANDOM_STATE
)
```

E a seguir verificamos a proporção dos tres conjuntos em termos de quantidade de reviews e quantidades de sequencias. Podemos observar que existe uma pequena diferença no balanceamos entre reviews e sequencias, o que é normal e esperado. Como a diferença apresentada entre ambos é pequena, podemos manter a separação dos conjuntos realizada.

In [18…

```
datasets = {
    "Treino": df_train,
    "Validação": df_val,
    "Teste": df_test,
}

total_reviews = len(df_reviews)
total_sequences = df_reviews["sequence_count"].sum()

print(f"Total de reviews:    {total_reviews:,}")
print(f"Total de sequências: {total_sequences:,}")

for name, dataset in datasets.items():
    review_count = len(dataset)
    sequence_count = dataset["sequence_count"].sum()

    print(f"\n{name}")
    print(
        f"Reviews:    {review_count:,} "
        f"({review_count / total_reviews:.2%})"
    )
    print(
        f"Sequências: {sequence_count:,} "
        f"({sequence_count / total_sequences:.2%})"
    )
    print(dataset["sequence_count"].describe())
```

```
Total de reviews:    3,000
Total de sequências: 714,333

Treino
Reviews:    2,400 (80.00%)
Sequências: 579,767 (81.16%)
count    2400.000000
mean      241.569583
std       187.418938
min        24.000000
25%       129.000000
50%       176.000000
75%       293.000000
max      2482.000000
Name: sequence_count, dtype: float64

Validação
Reviews:    600 (20.00%)
Sequências: 134,566 (18.84%)
count     600.000000
mean      224.276667
std       165.767875
min        30.000000
25%       126.000000
50%       164.500000
75%       264.250000
max      1003.000000
Name: sequence_count, dtype: float64

Teste
Reviews:    500 (16.67%)
Sequências: 115,241 (16.13%)
count     500.000000
mean      230.482000
std       177.759271
min        24.000000
25%       124.000000
50%       166.500000
75%       278.000000
max      1075.000000
Name: sequence_count, dtype: float64
```

2º Passo:

- Realize a indexação dos textos, utilizando a função `TextVectorization`:
  - Considere, *inicialmente*, um vocabulário de tamanho 5000;
  - Faça a indexação sem preenchimento (*padding*). Para isso, defina as variáveis `output_sequence_length = None` e `ragged = True`.
- Defina a lista de vocabulário por meio do método `get_vocabulary` do objeto de `TextVectorization`;
- Defina os dados indexados com o nome `token_ids`.

Pudemos observar no item 1 que o tamanho de `VOCAB_SIZE = 5_000` implica uma cobertura de cerca de 90% dos tokens das amostras que utilizaremos no treinamento. Vamos inicialmente criar três lista contendo os textos de cada conjunto: treinamento, validação e teste.

In [18…

```
texts_train = df_train["reviews"].tolist()
texts_val = df_val["reviews"].tolist()
texts_test = df_test["reviews"].tolist()
```

A seguir vamos criar nosso vetorizador de textos exclusivamente a partir dos textos do conjunto de treino.

In [18…

```
vectorizer = layers.TextVectorization(
    max_tokens=VOCAB_SIZE,
    standardize=Preprocessor,
    output_sequence_length=None,
    ragged=True,
    output_mode="int",
)

# Aprende o vocabulário exclusivamente com os textos de treino.
vectorizer.adapt(texts_train)

vocab = np.array(
    vectorizer.get_vocabulary()
)

vocab[:10]
```

Out[18…

```
array(['', '[UNK]', 'the', 'and', 'a', 'of', 'to', 'is', 'it', 'in'], dtype='<U16')
```

3º Passo:

- O *array* `token_ids` é formado por vetores indexados de tamanhos distintos. Mas, agora, iremos decompor cada vetor, *e.g.* $[2, 5, 3, 1, 8]$, como sequências do "próximo *token*":

$$[2, 5], [2, 5, 3], [2, 5, 3, 1], [2, 5, 3, 1, 8]$$

- Assim, defina uma lista `sequences` que irá capturar as sequências de **todos** os vetores;
- Utilize a função `pad_sequences` para preencher as sequências da esqueda para a direita:

$$[0, 0, 0, 2, 5], [0, 0, 2, 5, 3], [0, 2, 5, 3, 1], [2, 5, 3, 1, 8]$$

  - Defina o tamanho da entrada da rede `max_len = 16`;
  - Defina as variáveis da função `maxlen = max_len+1` e `padding = "pre"`.

Resolucao item 3

Antes de proceder com a resolução do item, é importante comentar e explicar as modificações que vamos implementar na sua proposição.

O problema com a abordagem de padding a esquerda sugerida é que ela impede a utilizacao de otimizações necessárias para processar corretamente o padding pela rede neural. Tentamos inicialmente utilizar a abordagem sugerida, porém o resultado gerou treinamento lento por não permitir utilizar algumas das possibilidades de otimização oferecidas pelo Keras. Em específico, no layer de Embeddings, necessitamos utilizar mask_zero=True para que o modelo ignore os paddings que vamos produzir. Porém, essa opção exige necessariamente padding a direita. Se habilitamos essa opção e utilizamos padding a esquerda, o Keras gera erro e não permite o processamento. E gerar os embeddings sem essa opção gera um modelo de pior qualidade.

Então vamos adotar uma abordagem inversa: queremos que as frases sofram padding à direita para permitir o uso da opção `mask_zero=True`. Com isso, vamos aproveitar e montar os arrays X e y numa unica etapa.

Considerando o exemplo dado no enunciado do exercício, as sequências de "próximo *token*" que adotaremos tem a seguinte forma:

Frase completa: $[2, 5, 3, 1, 8]$

Geração de sequências

Id	Sequência (X)	Próximo Token (y)
01	$[2, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]$	$[5]$
02	$[2, 5, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]$	$[3]$
03	$[2, 5, 3, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]$	$[1]$
04	$[2, 5, 3, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]$	$[8]$

É importante a notar também que as frases possuem tamanhos variáveis, como foi mostrado nos passos anteriores. Como vamos trabalhar com uma janela de contexto de 16 tokens, montar sequencias maiores que 16 tokens é disperdício de tempo e espaço. Vamos trabalhar então com uma janela deslizante de até 16 tokens sobre cada texto, descartando o primeiro token e produzindo um deslocamento à esquerda dos demais tokens para criar espaço para um eventual novo token. Podemos observar como isso vai funcionar no exemplo abaixo:

Frase completa: $[2, 5, 3, 1, 8, 35, 27, 86, 55, 10, 8, 5, 3, 12, 45, 14, 15, 10]$

Id	Sequência (X)	Próximo Token (y)	Observação
01	$[2, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]$	$[5]$	A janela começa com apenas 1 token
02	$[2, 5, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]$	$[3]$	e vai crescendo a medida que
03	$[2, 5, 3, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]$	$[1]$	novos tokens vao entrando na janela.
04	$[2, 5, 3, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]$	$[8]$	Todos os tokens a direita são [PAD] = 0
05	$[2, 5, 3, 1, 8, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]$	$[35]$	e vão desaparecendo na medida do crescimento
06	$[2, 5, 3, 1, 8, 35, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]$	$[27]$	da quantidade de tokens na janela;
07	$[2, 5, 3, 1, 8, 35, 27, 0, 0, 0, 0, 0, 0, 0, 0, 0]$	$[86]$	
08	$[2, 5, 3, 1, 8, 35, 27, 86, 0, 0, 0, 0, 0, 0, 0, 0]$	$[55]$	
09	$[2, 5, 3, 1, 8, 35, 27, 86, 55, 0, 0, 0, 0, 0, 0, 0]$	$[10]$	
10	$[2, 5, 3, 1, 8, 35, 27, 86, 55, 10, 0, 0, 0, 0, 0, 0]$	$[8]$	
11	$[2, 5, 3, 1, 8, 35, 27, 86, 55, 10, 8, 0, 0, 0, 0, 0]$	$[5]$	
12	$[2, 5, 3, 1, 8, 35, 27, 86, 55, 10, 8, 5, 0, 0, 0, 0]$	$[3]$	
13	$[2, 5, 3, 1, 8, 35, 27, 86, 55, 10, 8, 5, 3, 0, 0, 0]$	$[12]$	
14	$[2, 5, 3, 1, 8, 35, 27, 86, 55, 10, 8, 5, 3, 12, 0, 0]$	$[45]$	
15	$[2, 5, 3, 1, 8, 35, 27, 86, 55, 10, 8, 5, 3, 12, 45, 0]$	$[14]$	Quando a janela chega em
16	$[2, 5, 3, 1, 8, 35, 27, 86, 55, 10, 8, 5, 3, 12, 45, 14]$	$[15]$	tamanho 16
17	$[5, 3, 1, 8, 35, 27, 86, 55, 10, 8, 5, 3, 12, 45, 14, 15]$	$[10]$	A entrada de um novo token desloca todos os anteriores, eliminando o primeiro.

Outro ponto fundamental que foi notado no momento posterior de modelagem e avaliação dos resultados: treinar o modelo com sequências cuja previsão de próximo token fosse igual ao token desconhecido (`[UNK]`) não nos ajudaria a gerar as tres frases com conteúdo semanticamente significativo. Portanto, durante o processo de geração das sequencias definido a seguir, aquelas sequencias cujo próximo token previsto fossem iguais ao token `[UNK]` foram explicitamente ignoradas e excluídas da saída.

Por fim, criamos uma função que nos permitisse normalizar qualquer conjunto de textos, dado o vectorizer, os textos, o tamanho da janela e deixamos parametrizável se o token desconhecido deveria ser ignorado no target. Assim, foi possível aplicar sistematicamente ao conjunto de treino, validação e testes, bem como verificar o desempenho do modelo em relação a manter ou ignorar as sequências cuja previsão fosse `[UNK]`.

In [18…

```
def gerar_sequencias(
    texts,
    vectorizer,
    max_len,
    ignorar_target_unk=True
):
    token_ids = vectorizer(texts).to_list()

    contexts = []
    targets = []

    for document_ids in token_ids:
        for target_position in range(1, len(document_ids)):
            target = document_ids[target_position]

            # Ignora sequências cujo próximo token previsto seja o token desconhecido
            if ignorar_target_unk and target == 1:
                continue

            start_position = max(
                0,
                target_position - max_len
            )

            contexts.append(
                document_ids[start_position:target_position]
            )

            targets.append(
                target
            )

    X = pad_sequences(
        contexts,
        maxlen=max_len,
        padding="post",
        truncating="pre"
    )

    y = np.array(
        targets,
        dtype=np.int32
    )

    return X, y
```

4º Passo:

- Agora, iremos definir as variáveis X e y do modelo:
  - Onde X serão as sequências até o **penúltimo elemento**, *e.g.*, $[0, 0, 0, 2]$;
  - e y serão o **último elemento** das sequências, *e.g.*, $[5]$.
- Verifique se X apresenta dimensão `max_len`;

Resolução do item 4

Explicamos como vamos criar os conjuntos `X` e `y` no item anterior. E conforme explicado anteriormente, produzimos os conjuntos com padding à direita ao invés do proposto. A justificativa completa e detalhada da motivação que levou a trocar a abordagem sugerida está explicada na resolução do item 3.

A seguir executamos a função `gerar_sequencias` criada no item 3 para os conjuntos de treino, validação e testes.

In [18…

```
X_train, y_train = gerar_sequencias(
    texts_train,
    vectorizer,
    MAX_LEN
)

X_val, y_val = gerar_sequencias(
    texts_val,
    vectorizer,
    MAX_LEN
)

X_test, y_test = gerar_sequencias(
    texts_test,
    vectorizer,
    MAX_LEN
)
```

Em seguida mostramos o formato dos arrays de dados gerados.

In [18…

```
print("Treino:   ", X_train.shape, y_train.shape)
print("Validação:", X_val.shape, y_val.shape)
print("Teste:    ", X_test.shape, y_test.shape)
```

```
Treino:    (526774, 16) (526774,)
Validação: (120810, 16) (120810,)
Teste:     (103544, 16) (103544,)
```

E mostramos alguns exemplos de sequências e predições correspondentes geradas para o conjunto de treino.

In [18…

```
def imprimir_amostras_treino(
    X,
    y,
    quantidade=8,
    posicoes_iniciais=(0, 20),
    largura_linha=200
):
    np.set_printoptions(linewidth=largura_linha)

    print(X.shape)
    print(y.shape)

    for posicao_inicial in posicoes_iniciais:
        posicao_final = posicao_inicial + quantidade

        if posicao_inicial == 0:
            descricao = "1º exemplo"
        else:
            descricao = f"{posicao_inicial}º exemplo"

        print(
            f"\n======= {quantidade} exemplos "
            f"a partir do {descricao} ======"
        )

        print(f"Sequências:")
        print(X[posicao_inicial:posicao_final])
        print(f"Próximos tokens:")
        print(y[posicao_inicial:posicao_final])

imprimir_amostras_treino(
    X_train,
    y_train,
    quantidade=8,
    posicoes_iniciais=(0, 21)
)
```

```
(526774, 16)
(526774,)

======= 8 exemplos a partir do 1º exemplo ======
Sequências:
[[ 10   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0]
 [ 10  14   0   0   0   0   0   0   0   0   0   0   0   0   0   0]
 [ 10  14 163   0   0   0   0   0   0   0   0   0   0   0   0   0]
 [ 10  14 163 153   0   0   0   0   0   0   0   0   0   0   0   0]
 [ 10  14 163 153 152   0   0   0   0   0   0   0   0   0   0   0]
 [ 10  14 163 153 152  55   0   0   0   0   0   0   0   0   0   0]
 [ 10  14 163 153 152  55  11   0   0   0   0   0   0   0   0   0]
 [ 10  14 163 153 152  55  11 118   0   0   0   0   0   0   0   0]]
Próximos tokens:
[ 14 163 153 152  55  11 118  14]

======= 8 exemplos a partir do 21º exemplo ======
Sequências:
[[  14   22  238   31  195    8   14   60  441    2  151   69   56 3092    1    1]
 [  22  238   31  195    8   14   60  441    2  151   69   56 3092    1    1   14]
 [ 238   31  195    8   14   60  441    2  151   69   56 3092    1    1   14   40]
 [  31  195    8   14   60  441    2  151   69   56 3092    1    1   14   40 1776]
 [ 195    8   14   60  441    2  151   69   56 3092    1    1   14   40 1776   10]
 [   8   14   60  441    2  151   69   56 3092    1    1   14   40 1776   10   40]
 [  14   60  441    2  151   69   56 3092    1    1   14   40 1776   10   40  608]
 [  60  441    2  151   69   56 3092    1    1   14   40 1776   10   40  608    2]]
Próximos tokens:
[  14   40 1776   10   40  608    2  250]
```

Por fim queremos deixar registrado a função auxiliar que foi utilizada para identificar os alvos com predição de token desconhecido. No final os tres conjuntos ficaram sem previsão de targets `[UNK]`, mas sem o auxílio dessa função não seria possível verificar que nossa geração de sequências estava correta.

In [19…

```
UNK_ID = 1

def analisar_targets(y, nome):
    total = len(y)
    unk_count = np.sum(y == UNK_ID)

    print(nome)
    print(f"Total de sequências: {total:,}")
    print(f"Targets [UNK]:       {unk_count:,}")
    print(f"Proporção [UNK]:     {unk_count / total:.2%}")
    print()


analisar_targets(y_train, "Treino")
analisar_targets(y_val, "Validação")
analisar_targets(y_test, "Teste")
```

```
Treino
Total de sequências: 526,774
Targets [UNK]:       0
Proporção [UNK]:     0.00%

Validação
Total de sequências: 120,810
Targets [UNK]:       0
Proporção [UNK]:     0.00%

Teste
Total de sequências: 103,544
Targets [UNK]:       0
Proporção [UNK]:     0.00%
```

5º Passo:

- Desenvolva a rede neural:
  - Já sabemos do tamanho da camada de entrada, mas a **camada de saída** deverá *retornar qual o próximo token mais provável da lista de vocabulário*.
  - **Observação:** aqui, talvez, você ache incompatível a variável algo y com a camada de saída, mas iremos dar um jeito na função de custo, a seguir.

Resolução Item 5

Neste exercício decidimos adotar uma abordagem menos sistemática e mais exploratória que aquelas que adotamos nas atividades anteriores. Ao invés de trabalhar a partir de `GridSearch` e `RandomSearch` com espaços de busca potencialmente grandes, preferimos uma abordagem investigativa mais pequena, realizando modificações na modelagem um passo por vez. A cada nova modificação, avaliamos a solução comparando resultados quantitativos (na forma de função de perda e acurácia) e qualitativos (na forma de avaliação das frases geradas a partir do modelo). O principal motivador para modificar a abordagem foi que observamos que os treinamentos necessários no nosso contexto são potencialmente maiores e mais lentos que os existentes até então. Todo esse processo exploratório e iterativo será explicado no final da atividade.

No processo de modelagem chegamos a conclusão que seria benéfico para nossa atividade utilizar os embeddings pré-treinados do GloVe para atingir o objetivo de gerar 3 palavras em sequencia com significado. O processo que nos levou a tal conclusão será explicado no relatório do final da atividade. Também durante esse processo, notamos que aumentar o tamanho do embedding gerava resultados melhores. Porém, percebemos também que valeria mais a pena trabalhar a partir de um conjunto de embeddings pré-treinados. Como temos disponível os embeddings do GloVe com 100 dimensões, abandonamos os experimentos com embeddings diferentes de 100. Porém preservamos as principais características da modelagem que nos gerou melhor resultado.

In [19…

```
EMBED_DIM = 100 # para fazer transfer learning com o GloVe
```

Carregamos o glove a partir dos dados disponíveis localmente:

In [19…

```
GLOVE_PATH = "./dataset/glove.6B.100d.txt"

embeddings_index = {}

with open(GLOVE_PATH, encoding="utf-8") as f:
    for line in f:
        values = line.split()
        word = values[0]
        vector = np.asarray(values[1:], dtype="float32")
        embeddings_index[word] = vector

print(f"Vetores GloVe carregados: {len(embeddings_index)}")
```

```
Vetores GloVe carregados: 400000
```

A seguir montamos nossa matrix de embeddings a partir de uma inicialização aleatória. Isso nos permitirá tratar os tokens que não existem no GloVe de forma aleatória. Num primeiro momento deixamos os tokens zerados. Apesar da frequência baixa, entendemos que não permitir que o modelo gere diferenciação entre os tokens inexistentes no glove, ou pior, que eles sejam confundidos com padding, é mais prejudicial ao desempenho do modelo como um todo que deixar os tokens inexistentes com valores aleatórios.

Também aproveitamos para coletar algumas métricas para avaliar se a cobertura dos embeddings existentes no GloVe em relação aos nossos tokens poderia ser um problema.

In [19…

```
vocab = vectorizer.get_vocabulary()
num_tokens = len(vocab)

# vetor de inicialização aleatório para permitir que tokens ausentes sejam representados sem repetição de valores
embedding_matrix = np.random.normal(
    loc=0.0,
    scale=0.05,
    size=(num_tokens, EMBED_DIM)
).astype("float32")

# contadores para avaliar a cobertura dos embeddings existentes no GloVe em relação aos nossos tokens
hits = 0
misses = 0
missing_tokens = []

# para cada token, verificamos se ele existe no GloVe
for token_id, word in enumerate(vocab):
    embedding_vector = embeddings_index.get(word)

    if embedding_vector is not None:
        # se o token existe no GloVe, copiamos o vetor para a matrix de embeddings
        embedding_matrix[token_id] = embedding_vector
        hits += 1
    else:
        # se o token não existe no GloVe, adicionamos ele a lista de tokens ausentes
        misses += 1

        missing_tokens.append(
            (token_id, word)
        )

# Garantindo que somente o padding deve possuir vetor integralmente zerado.
embedding_matrix[0] = np.zeros(
    EMBED_DIM,
    dtype="float32"
)

print(f"Matrix de embeddings: {embedding_matrix.shape}")
```

```
Matrix de embeddings: (5000, 100)
```

Por fim contamos e verificamos quais os tokens não foram encontrados no GloVe para uma avaliação qualitativa. Foi essa verificação que nos permitiu identificar que a qualidade do nosso pré processamento deveria melhorar, considerando as contrações específicas mencionadas anteriormente.

In [19…

```
# Conta as ocorrências reais nos textos utilizados no treino.
token_ids_train = (
    vectorizer(texts_train)
    .flat_values
    .numpy()
)

frequencias = Counter(
    token_ids_train
)

print(f"Tokens encontrados no GloVe: {hits}")
print(f"Tokens não encontrados.....: {misses}")
print(f"Cobertura..................: {hits / num_tokens:.2%}")

print("\nTokens ausentes no GloVe:")
print(f"{'ID':>5}  {'TOKEN':<25} {'FREQUÊNCIA':>10}")
print("-" * 44)

for token_id, word in sorted(
    missing_tokens,
    key=lambda item: frequencias[item[0]],
    reverse=True
):
    token_exibido = (
        "<PADDING>"
        if token_id == 0
        else word
    )

    print(
        f"{token_id:>5}  "
        f"{token_exibido:<25} "
        f"{frequencias[token_id]:>10,}"
    )
```

```
Tokens encontrados no GloVe: 4994
Tokens não encontrados.....: 6
Cobertura..................: 99.88%

Tokens ausentes no GloVe:
   ID  TOKEN                     FREQUÊNCIA
--------------------------------------------
    1  [UNK]                         53,144
 3836  interferencia                     12
 4690  shepitko                           9
 4748  otoole                             9
 4823  hynkel                             9
    0  <PADDING>                          0
```

O modelo a seguir foi adaptado e readaptado a cada nova modificação da arquitetura. A cada nova iteração, as células eram limpas, o ambiente reiniciado e a nova execução era realizada. Deixamos comentados algumas das camadas que utilizamos no processo para fins de registro.

In [23…

```
x_in = layers.Input(
    shape=(MAX_LEN,), # entrada com o tamanho da janela MAX_LEN
    dtype=tf.int32,
    name="input")
x = layers.Embedding(
    VOCAB_SIZE,
    EMBED_DIM,
    weights=[embedding_matrix],
    mask_zero=True,
    trainable=False,
    name="embedding")(x_in)
# x = layers.Embedding(
#     input_dim=embedding_matrix.shape[0], # número de tokens
#     output_dim=embedding_matrix.shape[1], # dimensão dos embeddings
#     weights=[embedding_matrix], # inicialização com o GloVe
#     mask_zero=True, # para que o modelo ignore os tokens de padding
#     trainable=False, # para que os embeddings não sejam atualizados durante o treinamento
#     name="glove_embedding"
# )(x_in)
x = layers.LSTM(256, return_sequences=False, name="lstm")(x) # 256 unidades LSTM nos gerou melhores resultados que 128 ou outras opções menores
# x = layers.Bidirectional(layers.LSTM(384, return_sequences=True), name="bidirectional_lstm_1")(x)
# x = layers.Dropout(0.5, name="dropout")(x) # dropout para evitar overfitting
# x = layers.LSTM(384, return_sequences=True, name="lstm2")(x) # 256 unidades LSTM nos gerou melhores resultados que 128 ou outras opções menores
# x = layers.Dropout(0.5, name="dropout2")(x) # dropout para evitar overfitting
# x = layers.LSTM(384, return_sequences=False, name="lstm3")(x) # 256 unidades LSTM nos gerou melhores resultados que 128 ou outras opções menores
#x = layers.LSTM(128, return_sequences=False, name="lstm")(x)
x = layers.Dropout(0.5, name="dropout3")(x) # dropout para evitar overfitting
#x = layers.Dense(256, activation="relu", name="dense1")(x) # não gerou melhores resultados que a arquitetura sem essa camada
#x = layers.Dense(1600, activation="relu", name="dense3")(x) # não gerou melhores resultados que a arquitetura sem essa camada
x_out = layers.Dense(VOCAB_SIZE, activation="softmax", name="output")(x) # saída com o tamanho do vocabulário e softmax para gerar probabilidades

model = Model(x_in, x_out)
model.summary()
```

Model: "functional_5"

Layer (type)	Output Shape	Param #	Connected to
input (InputLayer)	(None, 16)	0	-
embedding (Embedding)	(None, 16, 100)	500,000	input[0][0]
not_equal_5 (NotEqual)	(None, 16)	0	input[0][0]
lstm (LSTM)	(None, 256)	365,568	embedding[0][0], not_equal_5[0][0]
dropout3 (Dropout)	(None, 256)	0	lstm[0][0]
output (Dense)	(None, 5000)	1,285,000	dropout3[0][0]

```
Total params: 2,150,568 (8.20 MB)
Trainable params: 1,650,568 (6.30 MB)
Non-trainable params: 500,000 (1.91 MB)
```

6º Passo:

- Compile o modelo com:
  - Otimizador `Adam`;
  - Para lidar com **saídas probabilística** e **variável alvo categórica**, utilizaremos a **função de custo** `sparse_categorical_crossentropy`;
  - Métrica `SparseTopKCategoricalAccuracy` para pegar os $k = 5$ maiores valores da camada de saída.

Resolução item 6

Durante o processo de modelagem exploratório iterativo tivemos algumas versões diferentes de configurações de otimizadores, learning rates, learning rate decay e de early stopping. A função a seguir representa a versão final que foi utilizada, com early stopping com patience = 4 épocas, e redução de learning rate por um fator de 0.5 com mínimo de lr = 1e-5 se não houver melhora por 2 épocas seguidas.

In [23…

```
callbacks = [
    EarlyStopping(
        monitor="val_loss",
        patience=4,
        restore_best_weights=True
    ),
    ReduceLROnPlateau(
        monitor="val_loss",
        factor=0.5,
        patience=2,
        min_lr=1e-5
    )
]
```

Da mesma forma, o learning rate inicial que produziu os melhores resultados foi de `lr = 3e-4`. A seguir compilamos o modelo utilizando as métricas sugeridas pelo exercício e as configurações mencionadas anteriormente.

In [23…

```
model.compile(
    optimizer=Adam(
        learning_rate=3e-4
    ),
    loss="sparse_categorical_crossentropy",
    metrics=[
        SparseTopKCategoricalAccuracy(
            k=5,
            name="top_5_accuracy"
        )
    ]
)
```

7º Passo:

- Realize o **treinamento validativo** do modelo com:
  - 15 épocas (inicialmente);
  - `batch_size` > 100;
  - `validation_split = 0.2`.

Resolução do item 7

A seguir executamos o treinamento. Ao longo da exploração realizada chegamos na conclusão que utilizar até 40 épocas para treinamento não seria um problema, considerando que temos o early stopping configurado, evitando o disperdicio de tempo. Também testamos alguns valores distintos de batch_size e chegamos no valor `batch_size=256` como nossa melhor escolha.

In [23…

```
history = model.fit(
    x=X_train,
    y=y_train,
    epochs=40,
    batch_size=256,
    validation_data=(
        X_val,
        y_val
    ),
    callbacks=callbacks
)
```

```
Epoch 1/40
2058/2058 ━━━━━━━━━━━━━━━━━━━━ 45s 22ms/step - loss: 6.2283 - top_5_accuracy: 0.2014 - val_loss: 5.9103 - val_top_5_accuracy: 0.2428 - learning_rate: 3.0000e-04
Epoch 2/40
2058/2058 ━━━━━━━━━━━━━━━━━━━━ 52s 25ms/step - loss: 5.8771 - top_5_accuracy: 0.2488 - val_loss: 5.6706 - val_top_5_accuracy: 0.2738 - learning_rate: 3.0000e-04
Epoch 3/40
2058/2058 ━━━━━━━━━━━━━━━━━━━━ 51s 25ms/step - loss: 5.7005 - top_5_accuracy: 0.2685 - val_loss: 5.5422 - val_top_5_accuracy: 0.2904 - learning_rate: 3.0000e-04
Epoch 4/40
2058/2058 ━━━━━━━━━━━━━━━━━━━━ 49s 24ms/step - loss: 5.5893 - top_5_accuracy: 0.2800 - val_loss: 5.4547 - val_top_5_accuracy: 0.2998 - learning_rate: 3.0000e-04
Epoch 5/40
2058/2058 ━━━━━━━━━━━━━━━━━━━━ 44s 21ms/step - loss: 5.5082 - top_5_accuracy: 0.2877 - val_loss: 5.3938 - val_top_5_accuracy: 0.3081 - learning_rate: 3.0000e-04
Epoch 6/40
2058/2058 ━━━━━━━━━━━━━━━━━━━━ 43s 21ms/step - loss: 5.4441 - top_5_accuracy: 0.2944 - val_loss: 5.3451 - val_top_5_accuracy: 0.3138 - learning_rate: 3.0000e-04
Epoch 7/40
2058/2058 ━━━━━━━━━━━━━━━━━━━━ 43s 21ms/step - loss: 5.3885 - top_5_accuracy: 0.2994 - val_loss: 5.3059 - val_top_5_accuracy: 0.3176 - learning_rate: 3.0000e-04
Epoch 8/40
2058/2058 ━━━━━━━━━━━━━━━━━━━━ 45s 22ms/step - loss: 5.3421 - top_5_accuracy: 0.3040 - val_loss: 5.2766 - val_top_5_accuracy: 0.3208 - learning_rate: 3.0000e-04
Epoch 9/40
2058/2058 ━━━━━━━━━━━━━━━━━━━━ 44s 21ms/step - loss: 5.3022 - top_5_accuracy: 0.3080 - val_loss: 5.2465 - val_top_5_accuracy: 0.3245 - learning_rate: 3.0000e-04
Epoch 10/40
2058/2058 ━━━━━━━━━━━━━━━━━━━━ 45s 22ms/step - loss: 5.2640 - top_5_accuracy: 0.3113 - val_loss: 5.2230 - val_top_5_accuracy: 0.3273 - learning_rate: 3.0000e-04
Epoch 11/40
2058/2058 ━━━━━━━━━━━━━━━━━━━━ 43s 21ms/step - loss: 5.2301 - top_5_accuracy: 0.3148 - val_loss: 5.2034 - val_top_5_accuracy: 0.3309 - learning_rate: 3.0000e-04
Epoch 12/40
2058/2058 ━━━━━━━━━━━━━━━━━━━━ 44s 21ms/step - loss: 5.1993 - top_5_accuracy: 0.3180 - val_loss: 5.1845 - val_top_5_accuracy: 0.3338 - learning_rate: 3.0000e-04
Epoch 13/40
2058/2058 ━━━━━━━━━━━━━━━━━━━━ 43s 21ms/step - loss: 5.1698 - top_5_accuracy: 0.3212 - val_loss: 5.1722 - val_top_5_accuracy: 0.3353 - learning_rate: 3.0000e-04
Epoch 14/40
2058/2058 ━━━━━━━━━━━━━━━━━━━━ 43s 21ms/step - loss: 5.1438 - top_5_accuracy: 0.3231 - val_loss: 5.1589 - val_top_5_accuracy: 0.3375 - learning_rate: 3.0000e-04
Epoch 15/40
2058/2058 ━━━━━━━━━━━━━━━━━━━━ 43s 21ms/step - loss: 5.1203 - top_5_accuracy: 0.3257 - val_loss: 5.1474 - val_top_5_accuracy: 0.3393 - learning_rate: 3.0000e-04
Epoch 16/40
2058/2058 ━━━━━━━━━━━━━━━━━━━━ 43s 21ms/step - loss: 5.0964 - top_5_accuracy: 0.3274 - val_loss: 5.1371 - val_top_5_accuracy: 0.3413 - learning_rate: 3.0000e-04
Epoch 17/40
2058/2058 ━━━━━━━━━━━━━━━━━━━━ 43s 21ms/step - loss: 5.0749 - top_5_accuracy: 0.3295 - val_loss: 5.1297 - val_top_5_accuracy: 0.3423 - learning_rate: 3.0000e-04
Epoch 18/40
```

```
2058/2058 ━━━━━━━━━━━━━━━━━━━━ 44s 21ms/step - loss: 5.0538 - top_5_accuracy: 0.3316 - val_loss: 5.1221 - val_top_5_accuracy: 0.3444 - learning_rate: 3.0000e-04
Epoch 19/40
2058/2058 ━━━━━━━━━━━━━━━━━━━━ 43s 21ms/step - loss: 5.0336 - top_5_accuracy: 0.3338 - val_loss: 5.1177 - val_top_5_accuracy: 0.3448 - learning_rate: 3.0000e-04
Epoch 20/40
2058/2058 ━━━━━━━━━━━━━━━━━━━━ 44s 21ms/step - loss: 5.0166 - top_5_accuracy: 0.3352 - val_loss: 5.1121 - val_top_5_accuracy: 0.3459 - learning_rate: 3.0000e-04
Epoch 21/40
2058/2058 ━━━━━━━━━━━━━━━━━━━━ 44s 21ms/step - loss: 4.9996 - top_5_accuracy: 0.3363 - val_loss: 5.1075 - val_top_5_accuracy: 0.3477 - learning_rate: 3.0000e-04
Epoch 22/40
2058/2058 ━━━━━━━━━━━━━━━━━━━━ 43s 21ms/step - loss: 4.9820 - top_5_accuracy: 0.3382 - val_loss: 5.1018 - val_top_5_accuracy: 0.3479 - learning_rate: 3.0000e-04
Epoch 23/40
2058/2058 ━━━━━━━━━━━━━━━━━━━━ 43s 21ms/step - loss: 4.9648 - top_5_accuracy: 0.3399 - val_loss: 5.0987 - val_top_5_accuracy: 0.3490 - learning_rate: 3.0000e-04
Epoch 24/40
2058/2058 ━━━━━━━━━━━━━━━━━━━━ 44s 22ms/step - loss: 4.9485 - top_5_accuracy: 0.3410 - val_loss: 5.0986 - val_top_5_accuracy: 0.3497 - learning_rate: 3.0000e-04
Epoch 25/40
2058/2058 ━━━━━━━━━━━━━━━━━━━━ 47s 23ms/step - loss: 4.9351 - top_5_accuracy: 0.3419 - val_loss: 5.0964 - val_top_5_accuracy: 0.3506 - learning_rate: 3.0000e-04
Epoch 26/40
2058/2058 ━━━━━━━━━━━━━━━━━━━━ 45s 22ms/step - loss: 4.9193 - top_5_accuracy: 0.3438 - val_loss: 5.0961 - val_top_5_accuracy: 0.3506 - learning_rate: 3.0000e-04
Epoch 27/40
2058/2058 ━━━━━━━━━━━━━━━━━━━━ 46s 22ms/step - loss: 4.9069 - top_5_accuracy: 0.3446 - val_loss: 5.0940 - val_top_5_accuracy: 0.3516 - learning_rate: 3.0000e-04
Epoch 28/40
2058/2058 ━━━━━━━━━━━━━━━━━━━━ 46s 22ms/step - loss: 4.8933 - top_5_accuracy: 0.3457 - val_loss: 5.0948 - val_top_5_accuracy: 0.3518 - learning_rate: 3.0000e-04
Epoch 29/40
2058/2058 ━━━━━━━━━━━━━━━━━━━━ 46s 22ms/step - loss: 4.8772 - top_5_accuracy: 0.3479 - val_loss: 5.0909 - val_top_5_accuracy: 0.3528 - learning_rate: 3.0000e-04
Epoch 30/40
2058/2058 ━━━━━━━━━━━━━━━━━━━━ 46s 22ms/step - loss: 4.8641 - top_5_accuracy: 0.3486 - val_loss: 5.0933 - val_top_5_accuracy: 0.3532 - learning_rate: 3.0000e-04
Epoch 31/40
2058/2058 ━━━━━━━━━━━━━━━━━━━━ 46s 22ms/step - loss: 4.8531 - top_5_accuracy: 0.3490 - val_loss: 5.0931 - val_top_5_accuracy: 0.3528 - learning_rate: 3.0000e-04
Epoch 32/40
2058/2058 ━━━━━━━━━━━━━━━━━━━━ 46s 22ms/step - loss: 4.8209 - top_5_accuracy: 0.3526 - val_loss: 5.0955 - val_top_5_accuracy: 0.3533 - learning_rate: 1.5000e-04
Epoch 33/40
2058/2058 ━━━━━━━━━━━━━━━━━━━━ 47s 23ms/step - loss: 4.8127 - top_5_accuracy: 0.3531 - val_loss: 5.0943 - val_top_5_accuracy: 0.3536 - learning_rate: 1.5000e-04
```

Funções auxiliares para análise da qualidade do modelo

Para permitir análise exploratória e iterativa, foi necessário fazer algumas verificações sobre os dados.

A primeira dessas verificações é ter idéia de quais os tokens previstos eram mais frequentes nos conjuntos de treino e validação, independentemente de qualquer treinamento. Em outras palavras, quais os tokens aparecem mais vezes nas variáveis `y_train` e `y_val`. Isso nos permite ter uma idéia do que seria um modelo de previsão baseado somente na frequência dos dados.

In [23…

```python
def exibir_targets_mais_frequentes(y, vocab, quantidade=10):
    ids, counts = np.unique(
        y,
        return_counts=True
    )

    ordem = np.argsort(
        counts
    )[::-1]

    total = len(y)

    for token_id, count in zip(
        ids[ordem][:quantidade],
        counts[ordem][:quantidade]
    ):
        print(
            f"{token_id:4d} "
            f"{vocab[token_id]:15s} "
            f"{count:6d} "
            f"{count / total:7.2%}"
        )


print("Conjunto de Treino")
exibir_targets_mais_frequentes(
    y_train,
    vocab
)

print("\nConjunto de Validação")
exibir_targets_mais_frequentes(
    y_val,
    vocab
)
```

```
Conjunto de Treino
   2 the              33130   6.29%
   3 and              16302   3.09%
   4 a                15943   3.03%
   5 of               14441   2.74%
   6 to               13336   2.53%
   7 is               13122   2.49%
   8 it                9430   1.79%
   9 in                9333   1.77%
  10 i                 8081   1.53%
  12 that              7103   1.35%

Conjunto de Validação
   2 the               7826   6.48%
   4 a                 3823   3.16%
   3 and               3742   3.10%
   5 of                3302   2.73%
   6 to                3220   2.67%
   7 is                3058   2.53%
   8 it                2146   1.78%
   9 in                2123   1.76%
  10 i                 1884   1.56%
  12 that              1679   1.39%
```

Em seguida, considerando que nossa métrica de acurácia considera os top 5 elementos da curva de probabilidade, queremos determinar o que seria um baseline mínimo de desempenho para o nosso modelo. Um modelo que somente prediz as top 5 palavras mais frequentes, quanto de acurácia obteria? Quais os tokens ele iria prever? O trecho de codigo a seguir responde isso: nosso modelo "ingênuo" deveria obter um baseline de pelo menos aproximadamente 18% de acurácia top 5.

In [24…

```python
def calcular_baseline_top_k(
    y_train,
    y_eval,
    k=5
):
    ids, counts = np.unique(
        y_train,
        return_counts=True
    )

    top_k_ids = ids[
        np.argsort(counts)[-k:]
    ]

    accuracy = np.mean(
        np.isin(y_eval, top_k_ids)
    )

    return top_k_ids, accuracy


top_5_ids, baseline_val = calcular_baseline_top_k(
    y_train,
    y_val,
    k=5
)

print("Tokens previstos pelo baseline:")
for token_id in top_5_ids:
    print(
        token_id,
        repr(vocab[token_id])
    )

print(
    f"\nBaseline top-5 na validação: "
    f"{baseline_val:.2%}"
)
```

```
Tokens previstos pelo baseline:
6 np.str_('to')
5 np.str_('of')
4 np.str_('a')
3 np.str_('and')
2 np.str_('the')

Baseline top-5 na validação: 18.14%
```

A seguinte função tem por objetivo comparar a probabilidade da próxima palavra gerada a partir de uma frase com a frequência global da respectiva palavra. Nossa hipótese com essa comparação é que, se o modelo está aprendendo adequadamente o contexto das palavras, a probabilidade da maioria das palavras geradas deveria divergir significativamente da frequência global da respectiva palavra.

In [24…

```python
def comparar_contexto_com_frequencia_global(
    modelo,
    texto,
    vectorizer,
    max_len,
    vocab,
    y_train,
    k=15
):
    ids = vectorizer([texto]).to_list()[0]

    x = pad_sequences(
        [ids[-max_len:]],
        maxlen=max_len,
        padding="post",
        truncating="pre"
    )

    probs = modelo.predict(
        x,
        verbose=0
    )[0].copy()

    probs[0] = 0  # padding
    probs[1] = 0  # [UNK]

    global_counts = np.bincount(
        y_train,
        minlength=len(vocab)
    )

    global_probs = (
        global_counts / global_counts.sum()
    )

    top_ids = np.argsort(
        probs
    )[-k:][::-1]

    print(f"\nContexto: {texto}\n")
    print(
        f"{'token':15s} "
        f"{'contexto':>10s} "
        f"{'global':>10s} "
        f"{'razão':>8s}"
    )

    for token_id in top_ids:
        contextual = probs[token_id]
        global_prob = global_probs[token_id]

        razao = (
            contextual / global_prob
            if global_prob > 0
            else np.inf
        )

        print(
            f"{vocab[token_id]:15s} "
            f"{contextual:9.3%} "
            f"{global_prob:9.3%} "
            f"{razao:8.2f}"
        )
```

In [24…

```python
# As frases de teste foram escolhidas para permitir uma avaliação mais ampla do modelo.
frases_teste = [
    "My opinion about the movie was that it was",
    "The actors in this move did a",
    "The best of music and the soundtrack was particularly",
    "what i liked the most in the movie",
    "The ending of the film reserved a",
]
```

In [24…

```python
for frase_original in frases_teste:

    comparar_contexto_com_frequencia_global(
        modelo=model,
        texto=frase_original,
        vectorizer=vectorizer,
        max_len=MAX_LEN,
        vocab=vocab,
        y_train=y_train,
        k=10
    )
```

```
Contexto: My opinion about the movie was that it was

token             contexto     global    razão
a                 14.373%    3.027%     4.75
not               11.617%    1.188%     9.78
the                4.915%    6.289%     0.78
so                 3.463%    0.399%     8.68
very               3.191%    0.244%    13.08
just               3.134%    0.338%     9.26
one                2.068%    0.524%     3.94
an                 2.015%    0.409%     4.93
really             1.921%    0.215%     8.93
made               1.423%    0.157%     9.07

Contexto: The actors in this move did a

token             contexto     global    razão
great              5.134%    0.171%    30.05
good               4.975%    0.271%    18.38
movie              4.335%    0.795%     5.45
film               3.606%    0.737%     4.89
little             2.438%    0.111%    21.99
few                2.415%    0.077%    31.41
lot                2.328%    0.070%    33.06
story              1.695%    0.215%     7.87
wonderful          1.539%    0.033%    46.86
couple             1.400%    0.040%    34.95

Contexto: The best of music and the soundtrack was particularly

token             contexto     global    razão
written            6.081%    0.030%   200.20
a                  5.057%    3.027%     1.67
very               4.808%    0.244%    19.71
the                3.557%    6.289%     0.57
done               2.080%    0.056%    37.28
great              1.754%    0.171%    10.26
good               1.468%    0.271%     5.42
in                 1.444%    1.772%     0.82
made               1.312%    0.157%     8.36
by                 1.253%    0.427%     2.93

Contexto: what i liked the most in the movie

token             contexto     global    razão
was               19.331%    0.950%    20.35
is                12.676%    2.491%     5.09
i                 10.202%    1.534%     6.65
and                4.428%    3.095%     1.43
of                 3.972%    2.741%     1.45
it                 2.905%    1.790%     1.62
in                 2.726%    1.772%     1.54
the                2.318%    6.289%     0.37
that               1.934%    1.348%     1.43
were               1.689%    0.207%     8.14

Contexto: The ending of the film reserved a

token             contexto     global    razão
few                4.687%    0.077%    60.96
couple             4.282%    0.040%   106.90
young              3.218%    0.074%    43.25
group              2.940%    0.024%   120.05
story              2.116%    0.215%     9.83
man                2.100%    0.119%    17.64
woman              2.032%    0.052%    39.07
country            1.622%    0.020%    82.14
girl               1.268%    0.049%    25.69
very               1.257%    0.244%     5.15
```

Colocando o modelo a prova

Embora somente no item 9 mais a frente se solicite explicitamente criar a função geradora dos textos, entendemos que devemos antecipar essa definição para permitir avaliar a qualidade do modelo, não apenas de forma quantitativa, mas também de forma qualitativa, na forma de verificação de coerencia das frases geradas. Portanto, definimos a seguir algumas frases que vamos utilizar para executar o processo de geração de palavras. E na sequência definimos duas estratégias distintas para gerar as palavras.

Gerador de Palavras Guloso

Primeiro definimos uma função de geração de palavras conforme a especificação da atividade. Geramos n palavras de forma iterativa, tomando cada palavra escolhida a partir daquela com maior probabilidade indicada pela saída do modelo e inserindo o token gerado no final da frase e voltando a iterar.

In [24…

```python
def gerar_proximas_palavras(
    modelo,
    texto,
    vectorizer,
    max_len,
    vocab,
    n=3
):
    """
    Gera as próximas palavras de forma determinística,
    selecionando sempre o token mais provável.
    """

    # Aplica o mesmo vectorizer utilizado durante o treinamento.
    ids = (
        vectorizer([texto])
        .to_list()[0]
    )

    novas = []

    for _ in range(n):
        # Preserva apenas os tokens mais recentes e aplica padding à direita.
        x = pad_sequences(
            [ids[-max_len:]],
            maxlen=max_len,
            padding="post",
            truncating="pre"
        )

        # Calcula a distribuição de probabilidades da próxima palavra.
        probs = modelo.predict(
            x,
            verbose=0
        )[0].copy()

        # Bloqueia tokens que não devem ser gerados.
        probs[0] = 0  # padding
        probs[1] = 0  # [UNK]

        # Seleciona a palavra mais provável.
        next_id = int(
            np.argmax(probs)
        )

        ids.append(next_id)
        novas.append(
            vocab[next_id]
        )

    return novas
```

Utilizando o gerador guloso

A seguir testamos o gerador com as frases de teste selecionadas anteriormente. Vamos tentar gerar 10 palavras (n=10), mas para efeitos de especulação do que propriamente tentar gerar frases completas. Para nosso objetivo, basta observar as três primeiras palavras do resultado.

In [25…

```python
for frase_original in frases_teste:
    novas = gerar_proximas_palavras(
        modelo=model,
        texto=frase_original,
        vectorizer=vectorizer,
        max_len=MAX_LEN,
        vocab=vocab,
        n=5
    )

    frase_gerada = (
        frase_original
        + " "
        + " ".join(novas)
    )

    print(f"Frase original: {frase_original}")
    print(f"Frase gerada:   {frase_gerada}")
    print("-" * 100)
```

```
Frase original: My opinion about the movie was that it was
Frase gerada:   My opinion about the movie was that it was a good movie i have
----------------------------------------------------------------------------------------------------
Frase original: The actors in this move did a
Frase gerada:   The actors in this move did a great film is a good
----------------------------------------------------------------------------------------------------
Frase original: The best of music and the soundtrack was particularly
Frase gerada:   The best of music and the soundtrack was particularly written and the actors are
----------------------------------------------------------------------------------------------------
Frase original: what i liked the most in the movie
Frase gerada:   what i liked the most in the movie was a good movie i
----------------------------------------------------------------------------------------------------
Frase original: The ending of the film reserved a
Frase gerada:   The ending of the film reserved a few years is a young
----------------------------------------------------------------------------------------------------
```

Gerador de Palavras Probabilístico

Decidimos também implementar uma versão de gerador de palavras probabilístico. Isto é, nosso gerador agora não toma a palavra de maior probabilidade, mas observa entre as top_k palavras de maior probabilidade, considerando uma penalidade por palavras repetidas para diminuir a repetição de palavras e uma temperatura, concentrando as propabilidades ou espalhando-as. Na pratica temperatura igual a 1 considera exatamente as probabilidades geradas pelo modelo. Temperatura menor que 1 torna a geração mais conservadora e temperatura maior que 1 deixa a geração mais "criativa", porém mais instável. Essa implementação vem na forma de duas funções: gerar_proximas_palavras_probabilistico e escolher_proximo_token:

In [24…

```python
def escolher_proximo_token(
    probs,
    ids_gerados,
    top_k=10,
    temperature=0.8,
    repetition_penalty=0.5,
    ultimos_n=5
):
    probs = probs.copy()

    # Não gerar padding nem [UNK].
    probs[0] = 0
    probs[1] = 0

    # Penaliza palavras usadas recentemente.
    for token_id in ids_gerados[-ultimos_n:]:
        probs[token_id] *= repetition_penalty

    # Evita log(0).
    probs = np.maximum(
        probs,
        1e-12
    )

    logits = (
        np.log(probs)
        / temperature
    )

    top_ids = np.argsort(
        logits
    )[-top_k:]

    top_logits = logits[
        top_ids
    ]

    top_probs = np.exp(
        top_logits
        - np.max(top_logits)
    )

    top_probs = (
        top_probs
        / top_probs.sum()
    )

    return int(
        np.random.choice(
            top_ids,
            p=top_probs
        )
    )
```

In [24…

```python
def gerar_proximas_palavras_probabilistico(
    modelo,
    texto,
    vectorizer,
    max_len,
    vocab,
    n=3,
    top_k=10,
    temperature=0.8,
    repetition_penalty=0.5
):
    ids = (
        vectorizer([texto])
        .to_list()[0]
    )

    novas = []

    for _ in range(n):
        x = pad_sequences(
            [ids[-max_len:]],
            maxlen=max_len,
            padding="post",
            truncating="pre"
        )

        probs = modelo.predict(
            x,
            verbose=0
        )[0]

        next_id = escolher_proximo_token(
            probs=probs,
            ids_gerados=ids,
            top_k=top_k,
            temperature=temperature,
            repetition_penalty=repetition_penalty
        )

        ids.append(
            next_id
        )

        novas.append(
            vocab[next_id]
        )

    return novas
```

Utilizando o gerador probabilístico

Com o uso dessas funções foi possível observar se a melhora quantitativa do modelo, em termos de acurácia e função de perda, se traduziam em melhoras perceptíveis na geração dos textos.

In [24…

```python
for frase_original in frases_teste:
    print(f"Frase original: {frase_original}")

    for _ in range(5):
        novas = gerar_proximas_palavras_probabilistico(
            modelo=model,
            texto=frase_original,
            vectorizer=vectorizer,
            max_len=MAX_LEN,
            vocab=vocab,
            n=6,
            top_k=8,
            temperature=0.8,
            repetition_penalty=0.6
        )

        frase_gerada = (
            frase_original
            + " "
            + " ".join(novas)
        )

        print(
            f"Frase gerada:   "
            f"{frase_gerada}"
        )

    print("-" * 100)
```

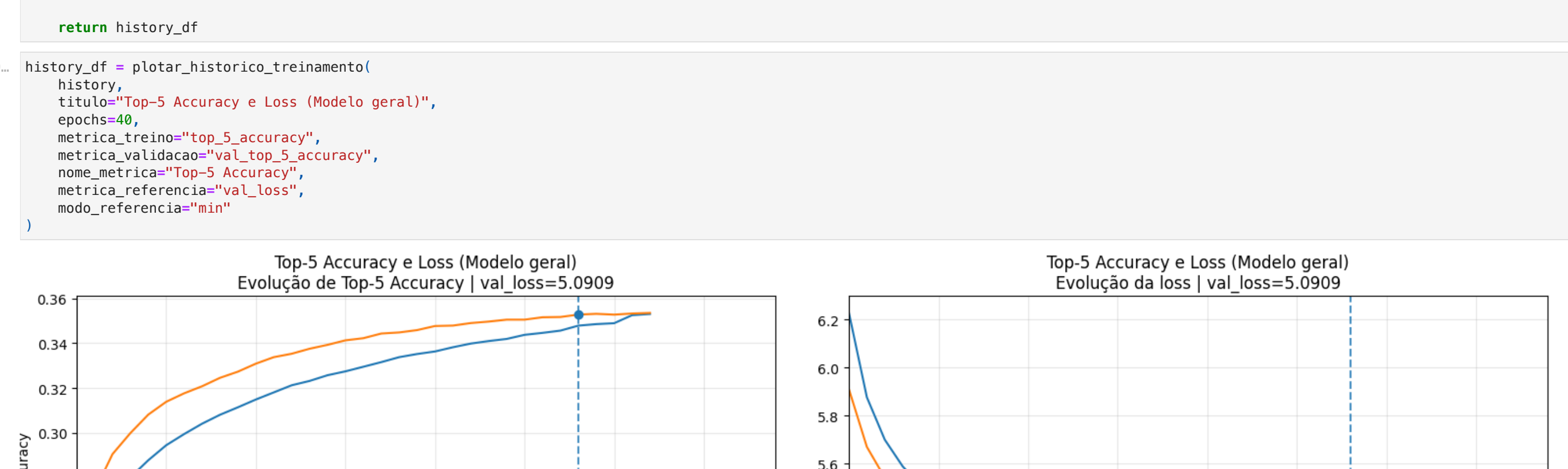
```
Frase original: My opinion about the movie was that it was
Frase gerada:   My opinion about the movie was that it was a good bad movie i have
Frase gerada:   My opinion about the movie was that it was not a good but i have
Frase gerada:   My opinion about the movie was that it was a good good film and not
Frase gerada:   My opinion about the movie was that it was one of the worst movies that
Frase gerada:   My opinion about the movie was that it was not a bad movie and the
----------------------------------------------------------------------------------------------------
Frase original: The actors in this move did a
Frase gerada:   The actors in this move did a good story on my opinion the
Frase gerada:   The actors in this move did a film is not a movie of
Frase gerada:   The actors in this move did a film is not a good
Frase gerada:   The actors in this move did a great piece of the film is
Frase gerada:   The actors in this move did a story about a young girl and
----------------------------------------------------------------------------------------------------
Frase original: The best of music and the soundtrack was particularly
Frase gerada:   The best of music and the soundtrack was particularly written by the perfect character of
Frase gerada:   The best of music and the soundtrack was particularly written the acting and a cast
Frase gerada:   The best of music and the soundtrack was particularly a good of the film i
Frase gerada:   The best of music and the soundtrack was particularly done and this one was a
Frase gerada:   The best of music and the soundtrack was particularly very good the movie is the
----------------------------------------------------------------------------------------------------
Frase original: what i liked the most in the movie
Frase gerada:   what i liked the most in the movie i was just looking to see
Frase gerada:   what i liked the most in the movie is very good i was surprised
Frase gerada:   what i liked the most in the movie was not a good movie i
Frase gerada:   what i liked the most in the movie was the only done in a
Frase gerada:   what i liked the most in the movie i had to see the movie
----------------------------------------------------------------------------------------------------
Frase original: The ending of the film reserved a
Frase gerada:   The ending of the film reserved a woman who gets in a small
Frase gerada:   The ending of the film reserved a woman is an cruel person of
Frase gerada:   The ending of the film reserved a story is one of those movies
Frase gerada:   The ending of the film reserved a few words in the last in
Frase gerada:   The ending of the film reserved a few years the characters are and
----------------------------------------------------------------------------------------------------
```

In [29…

```python
def preparar_historico_df(
    historico
):
    """
    Converte um histórico do Keras ou um DataFrame em DataFrame padronizado.

    Aceita:
    - objeto History retornado por model.fit();
    - pandas.DataFrame carregado de um history.csv.
    """
    if hasattr(historico, "history"):
        history_df = pd.DataFrame(
            historico.history
        )
    else:
        history_df = historico.copy()

    if "epoch" in history_df.columns:
        history_df["epoca"] = (
            history_df["epoch"] + 1
        )
    elif "epoca" not in history_df.columns:
        history_df["epoca"] = range(
            1,
            len(history_df) + 1
        )

    return history_df


def identificar_coluna_metrica(
    history_df,
    candidatos
):
    """
    Retorna a primeira coluna encontrada no DataFrame
    entre os nomes candidatos informados.
    """
    for coluna in candidatos:
        if coluna in history_df.columns:
            return coluna

    return None


def plotar_historico_treinamento(
    historico,
    titulo="Modelo",
    epochs=40,
    metrica_treino=None,
    metrica_validacao=None,
    nome_metrica="Métrica",
    metrica_referencia="val_loss",
    modo_referencia="min",
    intervalo_ticks=5
):
    """
    Plota a evolução da métrica principal e da loss de treino e validação.

    A melhor época é marcada com base em uma métrica de referência.
    Para este experimento, o padrão é val_loss, pois EarlyStopping,
    ReduceLROnPlateau e o objetivo do GridSearch usam val_loss.

    Parameters
    ----------
    historico : keras.callbacks.History ou pandas.DataFrame
        Histórico de treinamento.

    titulo : str
        Título base dos gráficos.

    epochs : int
        Limite máximo do eixo x.

    metrica_treino : str ou None
        Nome da coluna da métrica de treino.
        Exemplo: "top_5_accuracy".

    metrica_validacao : str ou None
        Nome da coluna da métrica de validação.
        Exemplo: "val_top_5_accuracy".

    nome_metrica : str
        Nome exibido no gráfico da métrica.

    metrica_referencia : str
        Métrica usada para marcar a melhor época.
        Exemplo: "val_loss" ou "val_top_5_accuracy".

    modo_referencia : str
        "min" para métricas que devem diminuir, como val_loss.
        "max" para métricas que devem aumentar, como val_top_5_accuracy.

    intervalo_ticks : int
        Intervalo entre os marcadores do eixo x.

    Returns
    -------
    pandas.DataFrame
        Histórico convertido para DataFrame.
    """
    history_df = preparar_historico_df(
        historico
    )

    if metrica_treino is None:
        metrica_treino = identificar_coluna_metrica(
            history_df,
            [
                "accuracy",
                "top_5_accuracy",
                "categorical_accuracy",
                "sparse_categorical_accuracy"
            ]
        )

    if metrica_validacao is None and metrica_treino is not None:
        metrica_validacao = f"val_{metrica_treino}"

    if (
        metrica_validacao is not None
        and metrica_validacao not in history_df.columns
    ):
        metrica_validacao = None

    if metrica_treino is None:
        raise ValueError(
            "Não foi encontrada uma coluna de métrica no histórico. "
            f"Colunas disponíveis: {list(history_df.columns)}"
        )

    if metrica_treino not in history_df.columns:
        raise ValueError(
            f"A métrica de treino '{metrica_treino}' não existe no histórico. "
            f"Colunas disponíveis: {list(history_df.columns)}"
        )

    if "loss" not in history_df.columns:
        raise ValueError(
            "A coluna 'loss' não existe no histórico. "
            f"Colunas disponíveis: {list(history_df.columns)}"
        )

    if metrica_referencia not in history_df.columns:
        raise ValueError(
            f"A métrica de referência '{metrica_referencia}' não existe "
            f"no histórico. Colunas disponíveis: {list(history_df.columns)}"
        )

    if modo_referencia == "min":
        melhor_indice = (
            history_df[metrica_referencia]
            .idxmin()
        )
    elif modo_referencia == "max":
        melhor_indice = (
            history_df[metrica_referencia]
            .idxmax()
        )
    else:
        raise ValueError(
            "modo_referencia deve ser 'min' ou 'max'."
        )

    melhor_epoca = int(
        history_df.loc[
            melhor_indice,
            "epoca"
        ]
    )

    melhor_valor_referencia = float(
        history_df.loc[
            melhor_indice,
            metrica_referencia
        ]
    )

    melhor_valor_metrica = None

    if metrica_validacao is not None:
        melhor_valor_metrica = float(
            history_df.loc[
                melhor_indice,
                metrica_validacao
            ]
        )

    xticks = list(
        range(
            1,
            epochs + 1,
            intervalo_ticks
        )
    )

    if epochs not in xticks:
        xticks.append(
            epochs
        )

    fig, axes = plt.subplots(
        nrows=1,
        ncols=2,
        figsize=(16, 5)
    )

    ax_metrica = axes[0]
    ax_loss = axes[1]

    ax_metrica.plot(
        history_df["epoca"],
        history_df[metrica_treino],
        label="Treino"
    )

    if metrica_validacao is not None:
        ax_metrica.plot(
            history_df["epoca"],
            history_df[metrica_validacao],
            label="Validação"
        )

    ax_metrica.axvline(
        melhor_epoca,
        linestyle="--",
        linewidth=1.5,
        alpha=0.8,
        label=f"Melhor época ({metrica_referencia}): {melhor_epoca}"
    )

    if melhor_valor_metrica is not None:
        ax_metrica.scatter(
            melhor_epoca,
            melhor_valor_metrica,
            zorder=3
        )

    ax_metrica.set_title(
        f"{titulo}\n"
        f"Evolução de {nome_metrica} | "
        f"{metrica_referencia}={melhor_valor_referencia:.4f}"
    )

    ax_metrica.set_xlabel("Época")
    ax_metrica.set_ylabel(nome_metrica)
    ax_metrica.set_xlim(1, epochs)
    ax_metrica.set_xticks(xticks)
    ax_metrica.grid(True, alpha=0.3)
    ax_metrica.legend()

    ax_loss.plot(
        history_df["epoca"],
        history_df["loss"],
        label="Treino"
    )

    if "val_loss" in history_df.columns:
        ax_loss.plot(
            history_df["epoca"],
            history_df["val_loss"],
            label="Validação"
        )

        ax_loss.scatter(
            melhor_epoca,
            melhor_valor_referencia,
            zorder=3
        )

    ax_loss.axvline(
        melhor_epoca,
        linestyle="--",
        linewidth=1.5,
        alpha=0.8,
        label=f"Melhor época ({metrica_referencia}): {melhor_epoca}"
    )

    ax_loss.set_title(
        f"{titulo}\n"
        f"Evolução da loss | "
        f"{metrica_referencia}={melhor_valor_referencia:.4f}"
    )

    ax_loss.set_xlabel("Época")
    ax_loss.set_ylabel("Loss")
    ax_loss.set_xlim(1, epochs)
    ax_loss.set_xticks(xticks)
    ax_loss.grid(True, alpha=0.3)
    ax_loss.legend()

    plt.tight_layout()
    plt.show()

    return history_df
```

In [29…

```python
history_df = plotar_historico_treinamento(
    history,
    titulo="Top-5 Accuracy e Loss (Modelo geral)",
    epochs=40,
    metrica_treino="top_5_accuracy",
    metrica_validacao="val_top_5_accuracy",
    nome_metrica="Top-5 Accuracy",
    metrica_referencia="val_loss",
    modo_referencia="min"
)
```



Uma última rodada de modelagem

Depois de executar o modelo exploratório iterativo mencionado anteriormente, ainda ficamos com algumas dúvidas quanto a como poderíamos melhorar o desempenho do modelo em termos de coerência das palavras geradas. Poderíamos investir em melhorar nossas funções geradoras de palavras, seja com estratégias probabilísticas mais sofisticadas, ou talvez com formas distintas de inferir a próxima palavra utilizando o modelo. Poderíamos também avançar em processar mais exemplos de palavras, considerando que o maior ganho marginal que obtivemos no processo iterativo veio exclusivamente do aumento de 500 para 2000 textos treinados. Porém, o fato de haver incorporado os embeddings pré-treinados do GloVe, numa tentativa de realizar transfer learning para melhorar nosso resultado, nos deixou com as seguintes perguntas em mente:

- Será que aumentar ainda mais os dados de treinamento, de 2000 para 3000 reviews, produziria resultados cada vez melhores ou atingiríamos um limite de desempenho? (spoiler: sim, existe melhora no resultado, por isso adotamos 3000 desde o início)
- Será que atualizar os pesos dos embeddings do GloVe durante o treinamento poderia produzir algum ajuste específico para o nosso contexto que os embeddings do GloVe puros não seriam capazes de reproduzir? (spoiler: embora o resultado quantitativo tenha melhorado, não notamos melhora significativa na análise qualitativa)
- Será que aumentar as camadas recorrentes produziriam melhores resultados? (spoiler: sim, aumentar as camadas recorrentes melhoraram um pouco o modelo, mas não a ponto de justificar o custo extra no tempo de processamento)

Para responder a primeira pergunta, basta aumentar a quantidade de reviews e comparar os resultados obtidos. Já para as duas últimas perguntas, decidimos montar um pequeno `GridSearch` com tão somente duas variáveis, dois valores em cada variável, produzindo 4 novas execuções, já com 3.000 exemplos de reviews no conjunto de treinamento. Então testamos o seguinte:

- Liberar o treinamento dos pesos dos embeddings do GloVe: `[ True, False ]`
- Camadas Recorrentes: `[ 256, 384 ]`

A seguir implementamos a modelagem mencionada.

In [26…

```python
class GridSearchComHistorico(
    kt.GridSearch
):
    """
    GridSearch que salva o histórico de cada trial em:
    trial_xxxx/history.csv
    """

    def run_trial(
        self,
        trial,
        *args,
        **kwargs
    ):
        trial_id = str(
            trial.trial_id
        )

        trial_dir = Path(
            self.get_trial_dir(
                trial_id
            )
        )

        trial_dir.mkdir(
            parents=True,
            exist_ok=True
        )

        callbacks_recebidos = kwargs.pop(
            "callbacks",
            []
        )

        callbacks_trial = list(
            callbacks_recebidos
        )

        callbacks_trial.append(
            tf.keras.callbacks.CSVLogger(
                str(
                    trial_dir / "history.csv"
                ),
                append=False
            )
        )

        kwargs["callbacks"] = callbacks_trial

        return super().run_trial(
            trial,
            *args,
            **kwargs
        )
```

In [27…

```python
RESULTS_BASE_DIR = Path("atividade4-results")
GRID_SEARCH_RESULTS_DIR = RESULTS_BASE_DIR / "grid-search"
GRID_SEARCH_PROJECT_NAME = "lstm-glove-grid-v1"

GRID_SEARCH_OVERWRITE = True
GRID_SEARCH_EPOCHS = 40
GRID_SEARCH_BATCH_SIZE = 256

GRID_SEARCH_OBJECTIVE = "val_loss"
GRID_SEARCH_OBJECTIVE_DIRECTION = "min"

GRID_SEARCH_GLOVE_TRAINABLE_VALUES = [
    False,
    True
]

GRID_SEARCH_LSTM_UNITS_VALUES = [
    256,
    384
]

MODEL_DROPOUT = 0.2
MODEL_LEARNING_RATE = 3e-4
MODEL_TOP_K_ACCURACY = 5

EARLY_STOPPING_MONITOR = "val_loss"
EARLY_STOPPING_PATIENCE = 4

REDUCE_LR_MONITOR = "val_loss"
REDUCE_LR_FACTOR = 0.5
REDUCE_LR_PATIENCE = 2
REDUCE_LR_MIN = 1e-5

GRID_SEARCH_RESULTS_DIR.mkdir(
    parents=True,
    exist_ok=True
)
```

In [27…

```python
def construir_modelo_grid_search(hp):
    glove_trainable = hp.Choice(
        "glove_trainable",
        values=GRID_SEARCH_GLOVE_TRAINABLE_VALUES
    )

    lstm_units = hp.Choice(
        "lstm_units",
        values=GRID_SEARCH_LSTM_UNITS_VALUES
    )

    x_in = layers.Input(
        shape=(MAX_LEN,),
        dtype=tf.int32,
        name="input"
    )

    x = layers.Embedding(
        input_dim=embedding_matrix.shape[0],
        output_dim=embedding_matrix.shape[1],
        weights=[embedding_matrix],
        mask_zero=True,
        trainable=glove_trainable,
        name="glove_embedding"
    )(x_in)

    x = layers.LSTM(
        lstm_units,
        return_sequences=False,
        name="lstm"
    )(x)

    x = layers.Dropout(
        MODEL_DROPOUT,
        name="dropout"
    )(x)

    x_out = layers.Dense(
        VOCAB_SIZE,
        activation="softmax",
        name="output"
    )(x)

    model = Model(
        x_in,
        x_out
    )

    model.compile(
        optimizer=Adam(
            learning_rate=MODEL_LEARNING_RATE
        ),
        loss="sparse_categorical_crossentropy",
        metrics=[
            SparseTopKCategoricalAccuracy(
                k=MODEL_TOP_K_ACCURACY,
                name="top_5_accuracy"
            )
        ]
    )

    return model
```

In [27…

```python
callbacks = [
    EarlyStopping(
        monitor=EARLY_STOPPING_MONITOR,
        patience=EARLY_STOPPING_PATIENCE,
        restore_best_weights=True
    ),
    ReduceLROnPlateau(
        monitor=REDUCE_LR_MONITOR,
        factor=REDUCE_LR_FACTOR,
        patience=REDUCE_LR_PATIENCE,
        min_lr=REDUCE_LR_MIN
    )
]

tuner = GridSearchComHistorico(
    hypermodel=construir_modelo_grid_search,
    objective=kt.Objective(
        GRID_SEARCH_OBJECTIVE,
        direction=GRID_SEARCH_OBJECTIVE_DIRECTION
    ),
    directory=str(
        GRID_SEARCH_RESULTS_DIR
    ),
    project_name=GRID_SEARCH_PROJECT_NAME,
    overwrite=GRID_SEARCH_OVERWRITE
)
```

In [27…

```python
tuner.search(
    x=X_train,
    y=y_train,
    validation_data=(
        X_val,
        y_val
    ),
    epochs=GRID_SEARCH_EPOCHS,
    batch_size=GRID_SEARCH_BATCH_SIZE,
    callbacks=callbacks,
    verbose=1
)
```

```
Trial 4 Complete [00h 14m 00s]
val_loss: 5.014655113220215

Best val_loss So Far: 5.014655113220215
Total elapsed time: 01h 12m 33s
```

In [27…

```python
GRID_SEARCH_SUMMARY_FILE = (
    RESULTS_BASE_DIR
    / "grid-search-summary.csv"
)

GRID_SEARCH_BEST_MODEL_FILE = (
    RESULTS_BASE_DIR
    / "best-model.keras"
)

GRID_SEARCH_NUM_TRIALS = (
    len(GRID_SEARCH_GLOVE_TRAINABLE_VALUES)
    * len(GRID_SEARCH_LSTM_UNITS_VALUES)
)
```

In [27…

```python
tuner.results_summary(
    num_trials=GRID_SEARCH_NUM_TRIALS
)
```

```
Results summary
Results in atividade4-results/grid-search/lstm-glove-grid-v1
Showing 4 best trials
Objective(name="val_loss", direction="min")

Trial 0003 summary
Hyperparameters:
glove_trainable: 1
lstm_units: 384
Score: 5.014655113220215

Trial 0002 summary
Hyperparameters:
glove_trainable: 1
lstm_units: 256
Score: 5.019923686981201

Trial 0001 summary
Hyperparameters:
glove_trainable: 0
lstm_units: 384
Score: 5.1084699630737305

Trial 0000 summary
Hyperparameters:
glove_trainable: 0
lstm_units: 256
Score: 5.111721038818359
```

In [27…

```python
def obter_valor_metrica_no_passo(
    trial,
    metric_name,
    step
):
    history = trial.metrics.get_history(
        metric_name
    )

    for observation in history:
        if observation.step == step:
            return observation.value[0]

    return np.nan


def criar_dataframe_resultados_grid_search(
    tuner
):
    resultados = []

    trials = tuner.oracle.get_best_trials(
        num_trials=GRID_SEARCH_NUM_TRIALS
    )

    for posicao, trial in enumerate(
        trials,
        start=1
    ):
        val_loss_history = trial.metrics.get_history(
            GRID_SEARCH_OBJECTIVE
        )

        melhor_observacao = min(
            val_loss_history,
            key=lambda observation: observation.value[0]
        )

        melhor_epoca = (
            melhor_observacao.step + 1
        )

        melhor_val_loss = (
            melhor_observacao.value[0]
        )

        val_top_5_accuracy = (
            obter_valor_metrica_no_passo(
                trial=trial,
                metric_name="val_top_5_accuracy",
                step=melhor_observacao.step
            )
        )

        modelo_trial = (
            construir_modelo_grid_search(
                trial.hyperparameters
            )
        )

        parametros_totais = (
            modelo_trial.count_params()
        )

        parametros_treinaveis = int(
            np.sum([
                np.prod(weight.shape)
                for weight
                in modelo_trial.trainable_weights
            ])
        )

        resultados.append({
            "posicao": posicao,
            "trial_id": trial.trial_id,
            "status": trial.status,
            "glove_trainable": (
                trial.hyperparameters.get(
                    "glove_trainable"
                )
            ),
            "lstm_units": (
                trial.hyperparameters.get(
                    "lstm_units"
                )
            ),
            "melhor_epoca": melhor_epoca,
            "val_loss": melhor_val_loss,
            "val_top_5_accuracy": (
                val_top_5_accuracy
            ),
            "parametros_totais": (
                parametros_totais
            ),
            "parametros_treinaveis": (
                parametros_treinaveis
            ),
        })

    return pd.DataFrame(
        resultados
    )
```

In [27…

```python
df_grid_search_results = (
    criar_dataframe_resultados_grid_search(
        tuner
    )
)

df_grid_search_results
```

Out[27…

	posicao	trial_id	status	glove_trainable	lstm_units	melhor_epoca	val_loss	val_top_5_accuracy	parametros_totais	parametros_treinaveis
0	1	0003	COMPLETED	1	384	14	5.014655	0.369853	3169960	3169960
1	2	0002	COMPLETED	1	256	20	5.019924	0.369241	2150568	2150568
2	3	0001	COMPLETED	0	384	15	5.108470	0.353141	3169960	2669960
3	4	0000	COMPLETED	0	256	25	5.111721	0.353398	2150568	1650568

In [27…

```python
df_grid_search_results.style.format({
    "val_loss": "{:.4f}",
    "val_top_5_accuracy": "{:.2%}",
    "parametros_totais": "{:,.0f}",
    "parametros_treinaveis": "{:,.0f}",
})
```

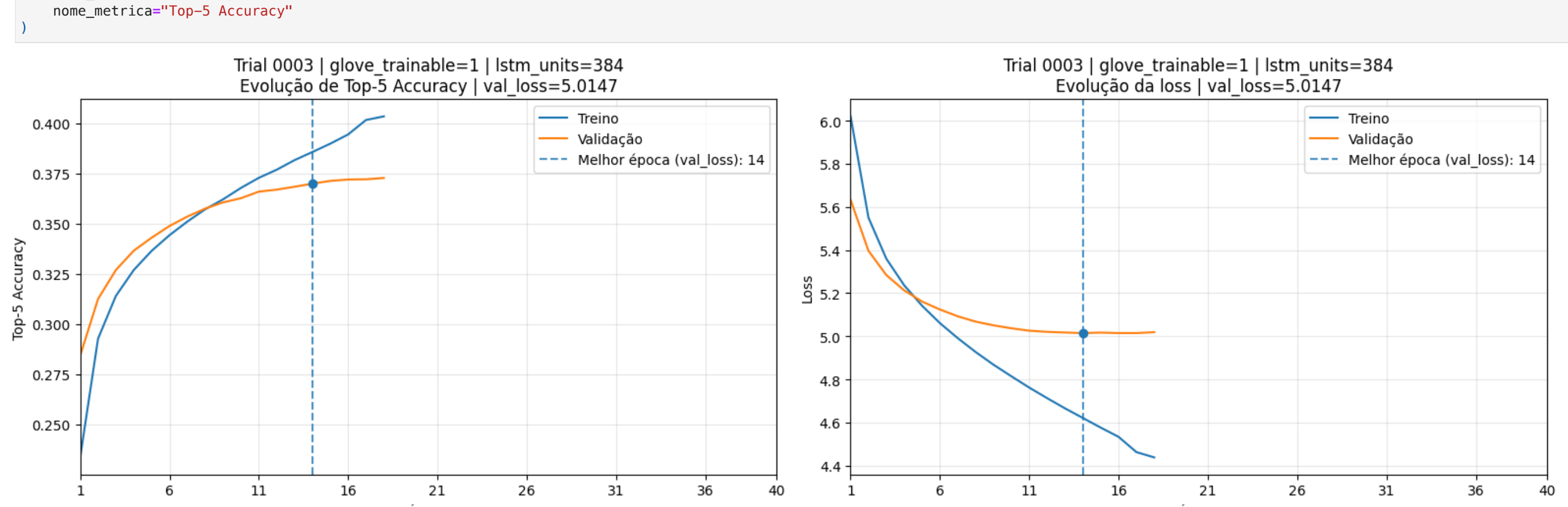
Out[27…

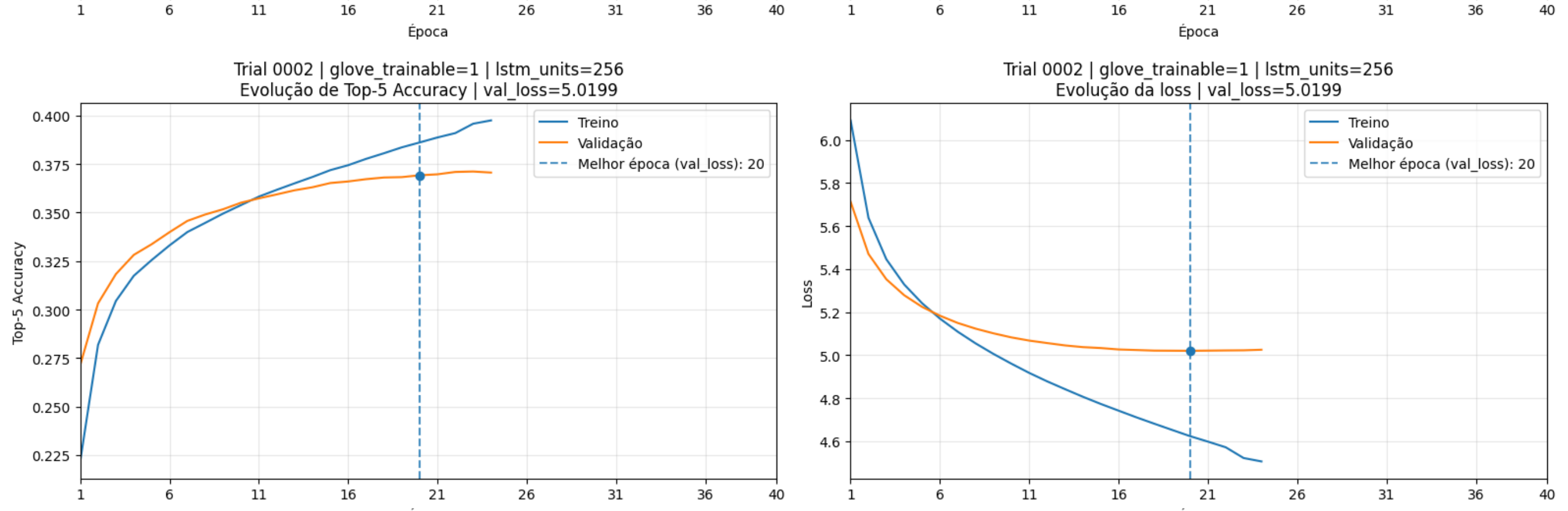
	posicao	trial_id	status	glove_trainable	lstm_units	melhor_epoca	val_loss	val_top_5_accuracy	parametros_totais	parametros_treinaveis
0	1	0003	COMPLETED	1	384	14	5.0147	36.99%	3,169,960	3,169,960
1	2	0002	COMPLETED	1	256	20	5.0199	36.92%	2,150,568	2,150,568
2	3	0001	COMPLETED	0	384	15	5.1085	35.31%	3,169,960	2,669,960
3	4	0000	COMPLETED	0	256	25	5.1117	35.34%	2,150,568	1,650,568

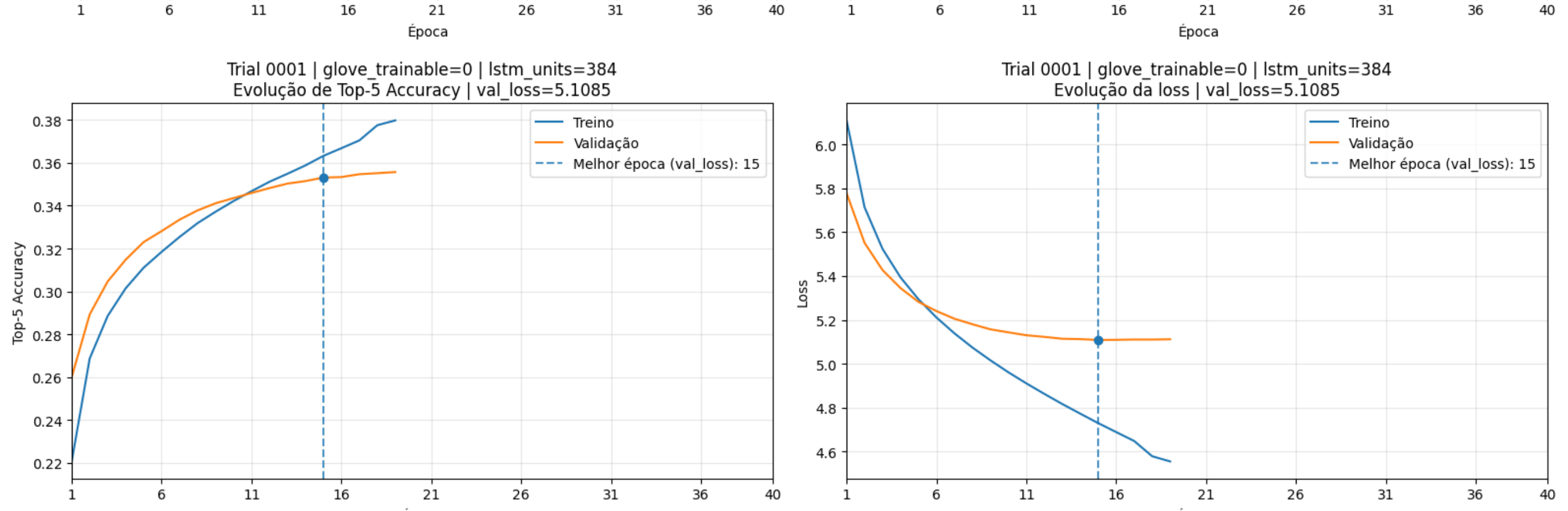
In [30…

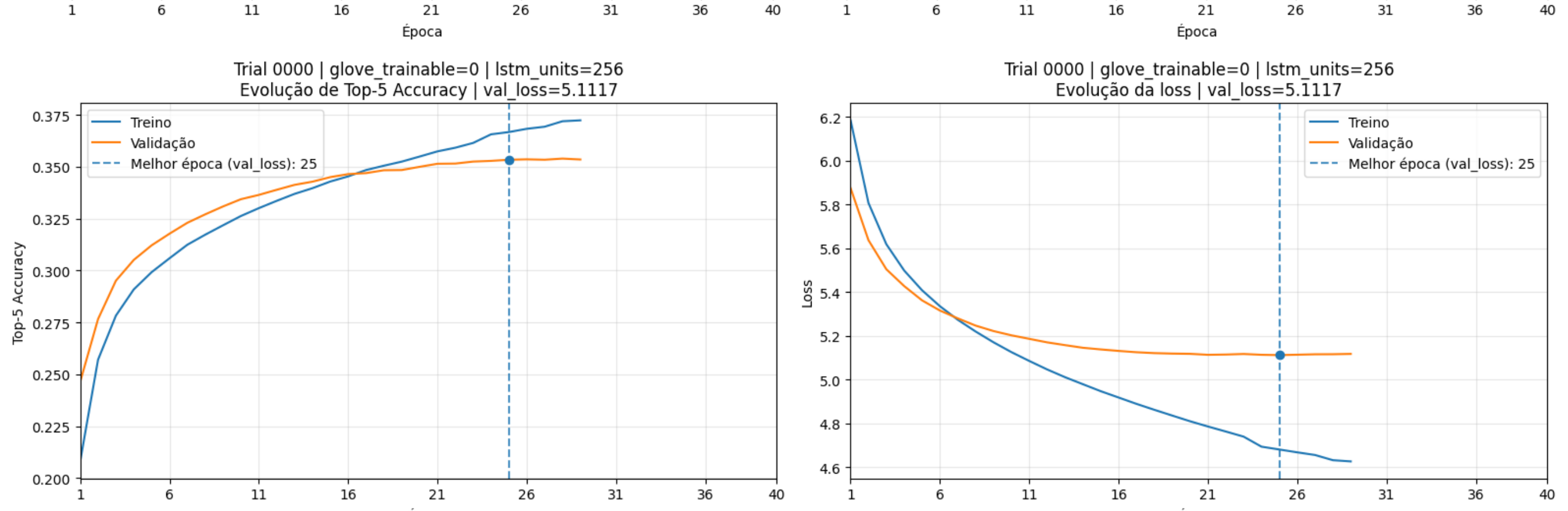
```python
def formatar_hiperparametros_trial(
    hiperparametros,
    max_caracteres=120
):
    """
    Formata os hiperparâmetros reais de um trial para exibição no título.

    Parameters
    ----------
    hiperparametros : dict
        Dicionário com os hiperparâmetros do trial.

    max_caracteres : int
        Limite de caracteres para evitar títulos muito longos.

    Returns
    -------
    str
        Texto formatado com os hiperparâmetros.
    """
    if not hiperparametros:
        return "hiperparâmetros não encontrados"

    partes = []

    for nome, valor in hiperparametros.items():
        if isinstance(valor, float):
            valor_formatado = f"{valor:.4f}"
        else:
            valor_formatado = str(valor)

        partes.append(
            f"{nome}={valor_formatado}"
        )

    texto = " | ".join(
        partes
    )

    if len(texto) > max_caracteres:
        texto = (
            texto[:max_caracteres]
            + "..."
        )

    return texto


def plotar_historico_grid_search(
    tuner,
    top_n=3,
    epochs=40,
    metrica_treino="top_5_accuracy",
    metrica_validacao="val_top_5_accuracy",
    metrica_referencia="val_loss",
    modo_referencia="min",
    nome_metrica="Top-5 Accuracy",
    nome_arquivo_historico="history.csv",
    ignorar_sem_historico=True
):
    """
    Carrega e plota os históricos dos melhores trials do tuner.

    Não realiza novo treinamento. A função depende dos históricos
    salvos em CSV dentro do diretório de cada trial.

    A melhor época é calculada com base em metrica_referencia.
    Para este experimento, o padrão correto é val_loss com modo min,
    pois EarlyStopping, ReduceLROnPlateau e GridSearch usam val_loss.

    Parameters
    ----------
    tuner : keras_tuner.Tuner
        Tuner previamente executado ou carregado com overwrite=False.

    top_n : int
        Quantidade de melhores trials que serão exibidos.

    epochs : int
        Limite máximo do eixo x.

    metrica_treino : str
        Nome da coluna da métrica de treino no history.csv.

    metrica_validacao : str
        Nome da coluna da métrica de validação no history.csv.

    metrica_referencia : str
        Métrica usada para identificar e marcar a melhor época.
        Exemplo: "val_loss".

    modo_referencia : str
        "min" para métricas que devem diminuir, como val_loss.
        "max" para métricas que devem aumentar, como val_top_5_accuracy.

    nome_metrica : str
        Nome exibido no gráfico.

    nome_arquivo_historico : str
        Nome do arquivo CSV com o histórico salvo no diretório do trial.

    ignorar_sem_historico : bool
        Se True, ignora trials sem history.csv.
        Se False, lança FileNotFoundError.

    Returns
    -------
    dict
        Históricos, hiperparâmetros, melhores épocas e métricas dos trials.
    """
    melhores_trials = tuner.oracle.get_best_trials(
        num_trials=top_n
    )

    historicos = {}
    trials_ignorados = []

    for trial in melhores_trials:
        trial_id = str(
            trial.trial_id
        )

        trial_dir = Path(
            tuner.get_trial_dir(
                trial_id
            )
        )

        history_file = (
            trial_dir
            / nome_arquivo_historico
        )

        if not history_file.exists():
            mensagem = (
                f"Histórico não encontrado para o trial {trial_id}: "
                f"{history_file}"
            )

            if ignorar_sem_historico:
                trials_ignorados.append(
                    {
                        "trial_id": trial_id,
                        "trial_dir": trial_dir,
                        "motivo": mensagem
                    }
                )

                continue

            raise FileNotFoundError(
                mensagem
            )

        history_df = pd.read_csv(
            history_file
        )

        history_df = preparar_historico_df(
            history_df
        )

        colunas_obrigatorias = [
            metrica_treino,
            metrica_referencia,
            "loss"
        ]

        colunas_ausentes = [
            coluna
            for coluna in colunas_obrigatorias
            if coluna not in history_df.columns
        ]

        if colunas_ausentes:
            raise ValueError(
                f"O histórico do trial {trial_id} não possui as colunas "
                f"necessárias: {colunas_ausentes}. "
                f"Colunas disponíveis: {list(history_df.columns)}"
            )

        if modo_referencia == "min":
            melhor_indice = (
                history_df[metrica_referencia]
                .idxmin()
            )
        elif modo_referencia == "max":
            melhor_indice = (
                history_df[metrica_referencia]
                .idxmax()
            )
        else:
            raise ValueError(
                "modo_referencia deve ser 'min' ou 'max'."
            )

        melhor_epoca = int(
            history_df.loc[
                melhor_indice,
                "epoca"
            ]
        )

        melhor_valor_referencia = float(
            history_df.loc[
                melhor_indice,
                metrica_referencia
            ]
        )

        melhor_valor_validacao = None

        if metrica_validacao in history_df.columns:
            melhor_valor_validacao = float(
                history_df.loc[
                    melhor_indice,
                    metrica_validacao
                ]
            )

        historicos[trial_id] = {
            "historico": history_df,
            "hiperparametros": trial.hyperparameters.values,
            "melhor_epoca": melhor_epoca,
            "melhor_valor_referencia": melhor_valor_referencia,
            "melhor_valor_validacao": melhor_valor_validacao,
            "metrica_referencia": metrica_referencia,
            "modo_referencia": modo_referencia,
            "trial_dir": trial_dir,
            "history_file": history_file
        }

    if not historicos:
        raise FileNotFoundError(
            "Nenhum histórico foi carregado. "
            "Provavelmente o grid search foi executado sem salvar history.csv "
            "com CSVLogger nos diretórios dos trials."
        )

    if trials_ignorados:
        print(
            "\nTrials ignorados por ausência de histórico:"
        )

        for item in trials_ignorados:
            print(
                f"- Trial {item['trial_id']}: {item['trial_dir']}"
            )

    for trial_id, resultado in historicos.items():
        hiperparametros_formatados = formatar_hiperparametros_trial(
            resultado["hiperparametros"]
        )

        titulo = (
            f"Trial {trial_id} | "
            f"{hiperparametros_formatados}"
        )

        plotar_historico_treinamento(
            resultado["historico"],
            titulo=titulo,
            epochs=epochs
            epochs=epochs,
            metrica_treino=metrica_treino,
            metrica_validacao=metrica_validacao,
            nome_metrica=nome_metrica,
            metrica_referencia=metrica_referencia,
            modo_referencia=modo_referencia
        )

    return {
        "historicos": historicos,
        "trials_ignorados": trials_ignorados
    }
```

In [30…
```
historicos_grid = plotar_historico_grid_search(
    tuner,
    top_n=4,
    epochs=GRID_SEARCH_EPOCHS,
    metrica_treino="top_5_accuracy",
    metrica_validacao="val_top_5_accuracy",
    metrica_referencia="val_loss",
    modo_referencia="min",
    nome_metrica="Top-5 Accuracy"
)
```









In [30…
```
best_hp = tuner.get_best_hyperparameters(
    num_trials=1
)[0]

print("Melhores hiperparâmetros:")
print(
    "glove_trainable:",
    best_hp.get("glove_trainable")
)
print(
    "lstm_units:",
    best_hp.get("lstm_units")
)
```

```
Melhores hiperparâmetros:
glove_trainable: 1
lstm_units: 384
```

In [28…
```
val_results = model.evaluate(
    X_val,
    y_val,
    batch_size=GRID_SEARCH_BATCH_SIZE,
    return_dict=True
)

print("Resultado na validação:")
print(val_results)
```

```
472/472 ━━━━━━━━━━━━━━━━━━━━ 4s 9ms/step - loss: 5.0909 - top_5_accuracy: 0.3528
Resultado na validação:
{'loss': 5.090921401977539, 'top_5_accuracy': 0.3528350293636322}
```

In [29…
```
N_WORDS_GREEDY = 5

for frase_original in frases_teste:
    novas = gerar_proximas_palavras(
        modelo=model,
        texto=frase_original,
        vectorizer=vectorizer,
        max_len=MAX_LEN,
        vocab=vocab,
        n=5
    )

    frase_gerada = (
        frase_original
        + " "
        + " ".join(novas)
    )

    print(
        f"Frase original: {frase_original}"
    )

    print(
        f"Frase gerada:   {frase_gerada}"
    )

    print("-" * 100)
```

```
Frase original: My opinion about the movie was that it was
Frase gerada:   My opinion about the movie was that it was a good movie i have
----------------------------------------------------------------------------------------------------
Frase original: The actors in this move did a
Frase gerada:   The actors in this move did a great film is a good
----------------------------------------------------------------------------------------------------
Frase original: The best of music and the soundtrack was particularly
Frase gerada:   The best of music and the soundtrack was particularly written and the actors are
----------------------------------------------------------------------------------------------------
Frase original: what i liked the most in the movie
Frase gerada:   what i liked the most in the movie was a good movie i
----------------------------------------------------------------------------------------------------
Frase original: The ending of the film reserved a
Frase gerada:   The ending of the film reserved a few years is a young
----------------------------------------------------------------------------------------------------
```

In [29…
```
N_WORDS_PROBABILISTIC = 5
N_GENERATED_SENTENCES = 5

PROBABILISTIC_TOP_K = 10
PROBABILISTIC_TEMPERATURE = 0.7
PROBABILISTIC_REPETITION_PENALTY = 0.4
```

In [29…
```
for frase_original in frases_teste:
    print(
        f"Frase original: {frase_original}"
    )

    for _ in range(
        N_GENERATED_SENTENCES
    ):
        novas = (
            gerar_proximas_palavras_probabilistico(
                modelo=model,
                texto=frase_original,
                vectorizer=vectorizer,
                max_len=MAX_LEN,
                vocab=vocab,
                n=N_WORDS_PROBABILISTIC,
                top_k=PROBABILISTIC_TOP_K,
                temperature=PROBABILISTIC_TEMPERATURE,
                repetition_penalty=(
                    PROBABILISTIC_REPETITION_PENALTY
                )
            )
        )

        frase_gerada = (
            frase_original
            + " "
            + " ".join(novas)
        )

        print(
            f"Frase gerada:   {frase_gerada}"
        )

    print("-" * 100)
```

```
Frase original: My opinion about the movie was that it was
Frase gerada:   My opinion about the movie was that it was a good movie but not
Frase gerada:   My opinion about the movie was that it was a little good movie and
Frase gerada:   My opinion about the movie was that it was a very good movie i
Frase gerada:   My opinion about the movie was that it was a great film the best
Frase gerada:   My opinion about the movie was that it was a good movie i thought
----------------------------------------------------------------------------------------------------
Frase original: The actors in this move did a
Frase gerada:   The actors in this move did a great example of all this
Frase gerada:   The actors in this move did a movie is a good film
Frase gerada:   The actors in this move did a good movie that is not
Frase gerada:   The actors in this move did a movie is the worst film
Frase gerada:   The actors in this move did a couple of the first episode
----------------------------------------------------------------------------------------------------
Frase original: The best of music and the soundtrack was particularly
Frase gerada:   The best of music and the soundtrack was particularly very far in the series
Frase gerada:   The best of music and the soundtrack was particularly made a few budget for
Frase gerada:   The best of music and the soundtrack was particularly written and the acting had
Frase gerada:   The best of music and the soundtrack was particularly written and the story was
Frase gerada:   The best of music and the soundtrack was particularly very good with the first
----------------------------------------------------------------------------------------------------
Frase original: what i liked the most in the movie
Frase gerada:   what i liked the most in the movie and i thought it was
Frase gerada:   what i liked the most in the movie was not a little good
Frase gerada:   what i liked the most in the movie i have watched it and
Frase gerada:   what i liked the most in the movie was so good i am
Frase gerada:   what i liked the most in the movie i would see this episode
----------------------------------------------------------------------------------------------------
Frase original: The ending of the film reserved a
Frase gerada:   The ending of the film reserved a few years is very good
Frase gerada:   The ending of the film reserved a couple of people are not
Frase gerada:   The ending of the film reserved a country is the worst movie
Frase gerada:   The ending of the film reserved a woman who is a young
Frase gerada:   The ending of the film reserved a young who has a woman
----------------------------------------------------------------------------------------------------
```

In [29…
```
BEST_MODEL_FILE = (
    RESULTS_BASE_DIR
    / "best-model.keras"
)

model.save(
    BEST_MODEL_FILE
)

print(
    "Melhor modelo salvo em:",
    BEST_MODEL_FILE
)
```

```
Melhor modelo salvo em: atividade4-results/best-model.keras
```

Escolhendo um modelo

Quando observamos o desempenho quantitativo dos 5 modelos, observamos que o trial 3 gerou os melhores resultados, com pouco mais de 1,5 p.p. superior. Isso mostra que aumentar a camada recorrente ajudou, assim como liberar o treinamento do GloVe. Entretanto, não foram utilizadas taxas de treinamento diferenciadamente pequenas para o treinamento dos embeddings. Isso pode levar a uma perda das relações semanticas. Embora as métricas tenham melhorado, elas podem ter vindo a um custo de aumento de overfitting. Na verdade, se observarmos as curvas de loss e acurácia para os modelos do GridSearch, podemos observar que existe overfitting. Porém ele foi controlado por conta do early stopping. Mas ele é mais pronunciado nos modelos com o treinamento do GloVe liberado. Por conta disso, vamos descartar a hipótese de liberar o treinamento do GloVe.

Quanto ao número de camadas recorrentes, podemos observar também que existe ganho quantitativo quando utilizamos camada LSTM de 384, embora muito menor que no caso anterior. Porém, o overfitting é maior no modelo com mais camadas recorrentes. Portanto, acreditamos que, embora existam ganhos numéricos perceptíveis, não detectamos uma melhora significativa na capacidade dos modelos. Isso nos leva a escolher a versão inicial que chamamos de "Modelo Geral" para seguir com nossa análise.

Para resumir, ficamos com a seguinte arquitetura:

Input (MAX_LEN,) int32 → **Embedding** VOCAB_SIZE× EMBED_DIM GloVe mask_zero=True trainable=False → **LSTM** 256 unidades return_sequences=False → **Dropout** rate = 0.5 → **Dense** VOCAB_SIZE softmax → **Saída** probabilidades do próximo token

In [30…
```
model.summary()
```

Model: "functional_5"

Layer (type)	Output Shape	Param #	Connected to
input (InputLayer)	(None, 16)	0	-
embedding (Embedding)	(None, 16, 100)	500,000	input[0][0]
not_equal_5 (NotEqual)	(None, 16)	0	input[0][0]
lstm (LSTM)	(None, 256)	365,568	embedding[0][0], not_equal_5[0][0]
dropout3 (Dropout)	(None, 256)	0	lstm[0][0]
output (Dense)	(None, 5000)	1,285,000	dropout3[0][0]

```
Total params: 5,451,706 (20.80 MB)
Trainable params: 1,650,568 (6.30 MB)
Non-trainable params: 500,000 (1.91 MB)
Optimizer params: 3,301,138 (12.59 MB)
```

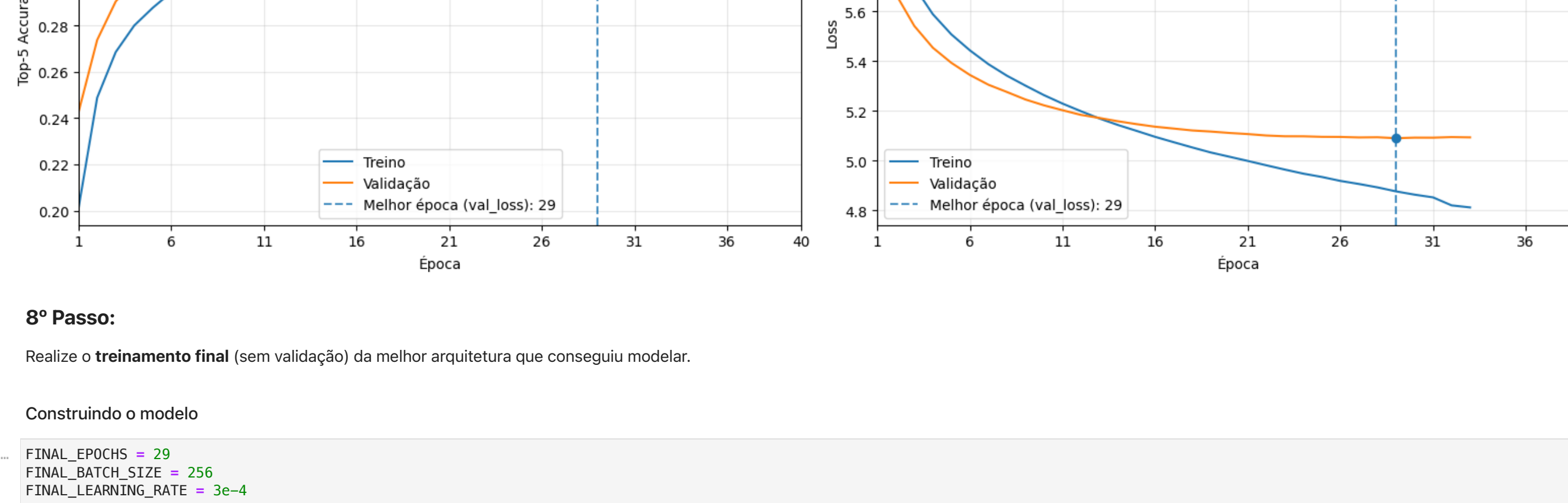
As frases geradas por esse modelo com o gerador guloso:

In [30…
```
for frase_original in frases_teste:
    novas = gerar_proximas_palavras(
        modelo=model,
        texto=frase_original,
        vectorizer=vectorizer,
        max_len=MAX_LEN,
        vocab=vocab,
        n=5
    )

    frase_gerada = (
        frase_original
        + " "
        + " ".join(novas)
    )

    print(f"Frase original: {frase_original}")
    print(f"Frase gerada:   {frase_gerada}")
    print("-" * 100)
```

```
Frase original: My opinion about the movie was that it was
Frase gerada:   My opinion about the movie was that it was a good movie i have
----------------------------------------------------------------------------------------------------
Frase original: The actors in this move did a
Frase gerada:   The actors in this move did a great film is a good
----------------------------------------------------------------------------------------------------
Frase original: The best of music and the soundtrack was particularly
Frase gerada:   The best of music and the soundtrack was particularly written and the actors are
----------------------------------------------------------------------------------------------------
Frase original: what i liked the most in the movie
Frase gerada:   what i liked the most in the movie was a good movie i
----------------------------------------------------------------------------------------------------
Frase original: The ending of the film reserved a
Frase gerada:   The ending of the film reserved a few years is a young
----------------------------------------------------------------------------------------------------
```

E as curvas de loss e acurácia:

In [30…
```
history_df = plotar_historico_treinamento(
    history,
    titulo="Top-5 Accuracy e Loss (Modelo geral)",
    epochs=40,
    metrica_treino="top_5_accuracy",
    metrica_validacao="val_top_5_accuracy",
    nome_metrica="Top-5 Accuracy",
    metrica_referencia="val_loss",
    modo_referencia="min"
)
```



8º Passo:

Realize o **treinamento final** (sem validação) da melhor arquitetura que conseguiu modelar.

Construindo o modelo

In [30…
```
FINAL_EPOCHS = 29
FINAL_BATCH_SIZE = 256
FINAL_LEARNING_RATE = 3e-4
FINAL_TOP_K = 5
FINAL_DROPOUT = 0.5
FINAL_LSTM_UNITS = 256
FINAL_GLOVE_TRAINABLE = False

FINAL_HISTORY_FILE = (
    RESULTS_BASE_DIR
    / "final-model-history.csv"
)

FINAL_TEST_RESULTS_FILE = (
    RESULTS_BASE_DIR
    / "final-model-test-results.csv"
)

FINAL_MODEL_FILE = (
    RESULTS_BASE_DIR
    / "modelo-final.keras"
)


def construir_modelo_final():
    """
    Constrói o modelo final escolhido para avaliação no conjunto de teste.

    Arquitetura:
    - Input com janela MAX_LEN;
    - Embedding inicializado com GloVe;
    - GloVe congelado;
    - LSTM com 256 unidades;
    - Dropout 0.5;
    - Dense softmax sobre o vocabulário.
    """
    x_in = layers.Input(
        shape=(MAX_LEN,),
        dtype=tf.int32,
        name="input"
    )

    x = layers.Embedding(
        input_dim=VOCAB_SIZE,
        output_dim=EMBED_DIM,
        weights=[embedding_matrix],
        mask_zero=True,
        trainable=FINAL_GLOVE_TRAINABLE,
        name="embedding"
    )(x_in)

    x = layers.LSTM(
        FINAL_LSTM_UNITS,
        return_sequences=False,
        name="lstm"
    )(x)

    x = layers.Dropout(
        FINAL_DROPOUT,
        name="dropout"
    )(x)

    x_out = layers.Dense(
        VOCAB_SIZE,
        activation="softmax",
        name="output"
    )(x)

    model = Model(
        x_in,
        x_out,
        name="modelo_final_lstm_glove"
    )

    model.compile(
        optimizer=Adam(
            learning_rate=FINAL_LEARNING_RATE
        ),
        loss="sparse_categorical_crossentropy",
        metrics=[
            SparseTopKCategoricalAccuracy(
                k=FINAL_TOP_K,
                name="top_5_accuracy"
            )
        ]
    )

    return model
```

Preparando dados de treino final

In [30…
```
X_train_final = np.concatenate(
    [
        X_train,
        X_val
    ],
    axis=0
)

y_train_final = np.concatenate(
    [
        y_train,
        y_val
    ],
    axis=0
)

print(
    "Formato de X_train_final:",
    X_train_final.shape
)

print(
    "Formato de y_train_final:",
    y_train_final.shape
)

print(
    "Formato de X_test:",
    X_test.shape
)

print(
    "Formato de y_test:",
    y_test.shape
)
```

```
Formato de X_train_final: (647584, 16)
Formato de y_train_final: (647584,)
Formato de X_test: (103544, 16)
Formato de y_test: (103544,)
```

Treinamento final do modelo

In [31…
```
tf.keras.backend.clear_session()

model_final = construir_modelo_final()

model_final.summary()

history_final = model_final.fit(
    X_train_final,
    y_train_final,
    epochs=FINAL_EPOCHS,
    batch_size=FINAL_BATCH_SIZE,
    callbacks=[
        CSVLogger(
            filename=str(
                FINAL_HISTORY_FILE
            ),
            append=False
        )
    ],
    verbose=1
)
```

Model: "modelo_final_lstm_glove"

Layer (type)	Output Shape	Param #	Connected to
input (InputLayer)	(None, 16)	0	-
embedding (Embedding)	(None, 16, 100)	500,000	input[0][0]
not_equal (NotEqual)	(None, 16)	0	input[0][0]
lstm (LSTM)	(None, 256)	365,568	embedding[0][0], not_equal[0][0]
dropout (Dropout)	(None, 256)	0	lstm[0][0]
output (Dense)	(None, 5000)	1,285,000	dropout[0][0]

```
Total params: 2,150,568 (8.20 MB)
Trainable params: 1,650,568 (6.30 MB)
Non-trainable params: 500,000 (1.91 MB)
Epoch 1/29
2530/2530 ━━━━━━━━━━━━━━━━━━━━ 49s 19ms/step - loss: 6.1739 - top_5_accuracy: 0.2084
Epoch 2/29
2530/2530 ━━━━━━━━━━━━━━━━━━━━ 47s 19ms/step - loss: 5.8049 - top_5_accuracy: 0.2570
Epoch 3/29
2530/2530 ━━━━━━━━━━━━━━━━━━━━ 48s 19ms/step - loss: 5.6308 - top_5_accuracy: 0.2770
Epoch 4/29
2530/2530 ━━━━━━━━━━━━━━━━━━━━ 48s 19ms/step - loss: 5.5227 - top_5_accuracy: 0.2880
Epoch 5/29
2530/2530 ━━━━━━━━━━━━━━━━━━━━ 48s 19ms/step - loss: 5.4435 - top_5_accuracy: 0.2957
Epoch 6/29
2530/2530 ━━━━━━━━━━━━━━━━━━━━ 48s 19ms/step - loss: 5.3806 - top_5_accuracy: 0.3021
Epoch 7/29
2530/2530 ━━━━━━━━━━━━━━━━━━━━ 48s 19ms/step - loss: 5.3291 - top_5_accuracy: 0.3075
Epoch 8/29
2530/2530 ━━━━━━━━━━━━━━━━━━━━ 48s 19ms/step - loss: 5.2858 - top_5_accuracy: 0.3111
Epoch 9/29
2530/2530 ━━━━━━━━━━━━━━━━━━━━ 48s 19ms/step - loss: 5.2468 - top_5_accuracy: 0.3157
Epoch 10/29
2530/2530 ━━━━━━━━━━━━━━━━━━━━ 49s 19ms/step - loss: 5.2112 - top_5_accuracy: 0.3192
Epoch 11/29
2530/2530 ━━━━━━━━━━━━━━━━━━━━ 49s 19ms/step - loss: 5.1822 - top_5_accuracy: 0.3215
Epoch 12/29
2530/2530 ━━━━━━━━━━━━━━━━━━━━ 48s 19ms/step - loss: 5.1541 - top_5_accuracy: 0.3249
Epoch 13/29
2530/2530 ━━━━━━━━━━━━━━━━━━━━ 50s 20ms/step - loss: 5.1278 - top_5_accuracy: 0.3275
Epoch 14/29
2530/2530 ━━━━━━━━━━━━━━━━━━━━ 48s 19ms/step - loss: 5.1032 - top_5_accuracy: 0.3296
Epoch 15/29
2530/2530 ━━━━━━━━━━━━━━━━━━━━ 49s 19ms/step - loss: 5.0807 - top_5_accuracy: 0.3319
Epoch 16/29
2530/2530 ━━━━━━━━━━━━━━━━━━━━ 48s 19ms/step - loss: 5.0604 - top_5_accuracy: 0.3337
Epoch 17/29
2530/2530 ━━━━━━━━━━━━━━━━━━━━ 49s 19ms/step - loss: 5.0390 - top_5_accuracy: 0.3357
Epoch 18/29
2530/2530 ━━━━━━━━━━━━━━━━━━━━ 48s 19ms/step - loss: 5.0215 - top_5_accuracy: 0.3379
Epoch 19/29
2530/2530 ━━━━━━━━━━━━━━━━━━━━ 48s 19ms/step - loss: 5.0035 - top_5_accuracy: 0.3389
Epoch 20/29
2530/2530 ━━━━━━━━━━━━━━━━━━━━ 48s 19ms/step - loss: 4.9877 - top_5_accuracy: 0.3411
Epoch 21/29
2530/2530 ━━━━━━━━━━━━━━━━━━━━ 48s 19ms/step - loss: 4.9699 - top_5_accuracy: 0.3426
Epoch 22/29
2530/2530 ━━━━━━━━━━━━━━━━━━━━ 48s 19ms/step - loss: 4.9561 - top_5_accuracy: 0.3440
Epoch 23/29
2530/2530 ━━━━━━━━━━━━━━━━━━━━ 48s 19ms/step - loss: 4.9424 - top_5_accuracy: 0.3452
Epoch 24/29
2530/2530 ━━━━━━━━━━━━━━━━━━━━ 48s 19ms/step - loss: 4.9275 - top_5_accuracy: 0.3463
Epoch 25/29
2530/2530 ━━━━━━━━━━━━━━━━━━━━ 48s 19ms/step - loss: 4.9147 - top_5_accuracy: 0.3474
Epoch 26/29
2530/2530 ━━━━━━━━━━━━━━━━━━━━ 55s 22ms/step - loss: 4.9033 - top_5_accuracy: 0.3489
Epoch 27/29
2530/2530 ━━━━━━━━━━━━━━━━━━━━ 48s 19ms/step - loss: 4.8876 - top_5_accuracy: 0.3505
Epoch 28/29
2530/2530 ━━━━━━━━━━━━━━━━━━━━ 48s 19ms/step - loss: 4.8775 - top_5_accuracy: 0.3513
Epoch 29/29
2530/2530 ━━━━━━━━━━━━━━━━━━━━ 48s 19ms/step - loss: 4.8659 - top_5_accuracy: 0.3528
```

Inferência com o conjunto de testes

In [31…
```
resultado_teste = model_final.evaluate(
    X_test,
    y_test,
    batch_size=FINAL_BATCH_SIZE,
    return_dict=True,
    verbose=1
)

resultado_teste_df = pd.DataFrame(
    [
        resultado_teste
    ]
)

resultado_teste_df.to_csv(
    FINAL_TEST_RESULTS_FILE,
    index=False
)

print(
    "\n=== Avaliação final no conjunto de teste ==="
)

for metrica, valor in resultado_teste.items():
    print(
        f"{metrica}: {valor:.6f}"
    )

print(
    f"\nResultado do teste salvo em: {FINAL_TEST_RESULTS_FILE}"
)
```

```
405/405 ━━━━━━━━━━━━━━━━━━━━ 4s 10ms/step - loss: 5.0609 - top_5_accuracy: 0.3531

=== Avaliação final no conjunto de teste ===
loss: 5.060905
top_5_accuracy: 0.353125

Resultado do teste salvo em: atividade4-results/final-model-test-results.csv
```

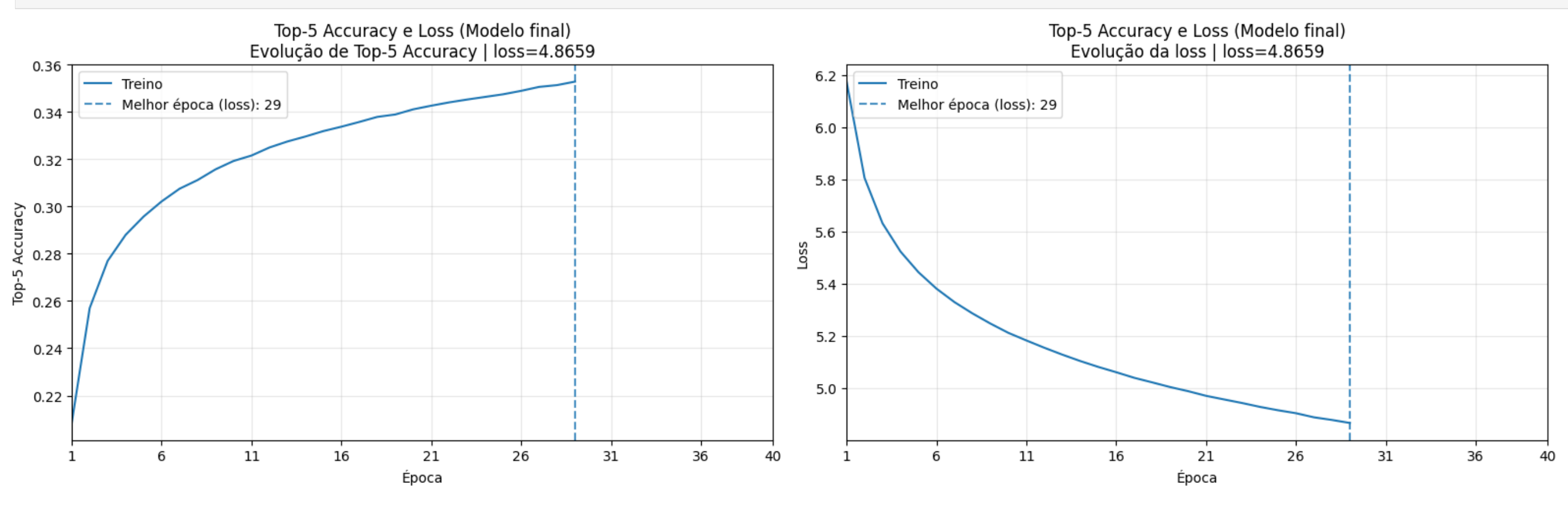
Salvando o modelo

In [31…
```
model_final.save(
    FINAL_MODEL_FILE
)

print(
    f"Modelo final salvo em: {FINAL_MODEL_FILE}"
)
```

```
Modelo final salvo em: atividade4-results/modelo-final.keras
```

Montando curva de Acurácia e Loss

In [31…
```
history_final_df = plotar_historico_treinamento(
    history_final,
    titulo="Top-5 Accuracy e Loss (Modelo final)",
    epochs=40,
    metrica_treino="top_5_accuracy",
    metrica_validacao=None,
    nome_metrica="Top-5 Accuracy",
    metrica_referencia="loss",
    modo_referencia="min"
)
```



9º Passo:

- Defina uma função para **gerar** o próximo *token* dado uma entrada de texto:
  - O texto deve entrar na forma de *string*, *e.g.*, "the movie was", então a função deverá utilizar o objeto de `TextVectorization`, já definido, para obter os tokens indexados;
  - Em seguinda, o *array* de tokens indexados deverá passar pelo `pad_sequences`, com tamanho `max_len`;
  - Realize a predição do modelo e utilize a função `argmax` do `Numpy` para encontrar o índice do *token* mais provável;
  - Com este índice, use a lista de vocabulário para encontrar o *token* mais provável;
  - Anexe este token à sentença de entrada;
  - Coloque toda a estrutura de código dos itens acima em um *loop*, a fim de gerar as n palavras mais provaveis.
- Seu modelo precisa completar ao menos **três palavras coerentes** com a frase de entrada.

Resolução do item 9

Vamos utilizar as funções geradoras criadas anteriormente.

Gerador guloso

In [31…
```
N_WORDS_GREEDY = 10

for frase_original in frases_teste:
    novas = gerar_proximas_palavras(
        modelo=model_final,
        texto=frase_original,
        vectorizer=vectorizer,
        max_len=MAX_LEN,
        vocab=vocab,
        n=3
    )

    frase_gerada = (
        frase_original
        + " "
        + " ".join(novas)
    )

    print(
        f"Frase original: {frase_original}"
    )

    print(
        f"Frase gerada:   {frase_gerada}"
    )

    print("-" * 100)
```

```
Frase original: My opinion about the movie was that it was
Frase gerada:   My opinion about the movie was that it was a good movie
----------------------------------------------------------------------------------------------------
Frase original: The actors in this move did a
Frase gerada:   The actors in this move did a lot of the
----------------------------------------------------------------------------------------------------
Frase original: The best of music and the soundtrack was particularly
Frase gerada:   The best of music and the soundtrack was particularly the acting was
----------------------------------------------------------------------------------------------------
Frase original: what i liked the most in the movie
Frase gerada:   what i liked the most in the movie i have seen
----------------------------------------------------------------------------------------------------
Frase original: The ending of the film reserved a
Frase gerada:   The ending of the film reserved a young woman who
----------------------------------------------------------------------------------------------------
```

Gerador Probabilistico

In [31…

```
for frase_original in frases_teste:
    print(
        f"Frase original: {frase_original}"
    )

    for _ in range(
        N_GENERATED_SENTENCES
    ):
        novas = (
            gerar_proximas_palavras_probabilistico(
                modelo=model_final,
                texto=frase_original,
                vectorizer=vectorizer,
                max_len=MAX_LEN,
                vocab=vocab,
                n=N_WORDS_PROBABILISTIC,
                top_k=PROBABILISTIC_TOP_K,
                temperature=PROBABILISTIC_TEMPERATURE,
                repetition_penalty=(
                    PROBABILISTIC_REPETITION_PENALTY
                )
            )
        )

        frase_gerada = (
            frase_original
            + " "
            + " ".join(novas)
        )

        print(
            f"Frase gerada:   {frase_gerada}"
        )

    print("-" * 100)
```

```
Frase original: My opinion about the movie was that it was
Frase gerada:   My opinion about the movie was that it was not made i have seen
Frase gerada:   My opinion about the movie was that it was a good movie i have
Frase gerada:   My opinion about the movie was that it was a bad movie is not
Frase gerada:   My opinion about the movie was that it was a good movie in its
Frase gerada:   My opinion about the movie was that it was not a bad film was
----------------------------------------------------------------------------------------------------
Frase original: The actors in this move did a
Frase gerada:   The actors in this move did a movie is so good it
Frase gerada:   The actors in this move did a bad movie was awful is
Frase gerada:   The actors in this move did a movie was just supposed to
Frase gerada:   The actors in this move did a good film is not the
Frase gerada:   The actors in this move did a good movie is a wonderful
----------------------------------------------------------------------------------------------------
Frase original: The best of music and the soundtrack was particularly
Frase gerada:   The best of music and the soundtrack was particularly bad i am not the
Frase gerada:   The best of music and the soundtrack was particularly good that i can not
Frase gerada:   The best of music and the soundtrack was particularly made and i can say
Frase gerada:   The best of music and the soundtrack was particularly bad i have seen this
Frase gerada:   The best of music and the soundtrack was particularly acted but really it is
----------------------------------------------------------------------------------------------------
Frase original: what i liked the most in the movie
Frase gerada:   what i liked the most in the movie i think of it is
Frase gerada:   what i liked the most in the movie that i am not not
Frase gerada:   what i liked the most in the movie that i am not a
Frase gerada:   what i liked the most in the movie i have seen i am
Frase gerada:   what i liked the most in the movie i have seen i am
----------------------------------------------------------------------------------------------------
Frase original: The ending of the film reserved a
Frase gerada:   The ending of the film reserved a man and has to be
Frase gerada:   The ending of the film reserved a man in the first time
Frase gerada:   The ending of the film reserved a woman who is an adult
Frase gerada:   The ending of the film reserved a country of the most exciting
Frase gerada:   The ending of the film reserved a woman who is one of
----------------------------------------------------------------------------------------------------
```

Evolução da modelagem para previsão das próximas palavras

O objetivo do experimento é construir uma rede neural recorrente capaz de receber uma sequência textual e gerar pelo menos três palavras adicionais com significado relevante para o contexto apresentado. Para isso, iniciamos utilizando 500 reviews aleatórias do dataset IMDB. A partir de cada review, foram geradas sequências de treinamento em que uma janela com até 16 tokens anteriores foi utilizada para prever o token seguinte.

A arquitetura inicial utilizou uma camada `Embedding`, uma camada `LSTM`, uma camada densa intermediária e uma camada de saída com ativação `softmax`. Inicialmente, o vocabulário máximo foi de 15.000 tokens, o embedding possuía 100 dimensões, a LSTM possuía 256 unidades e a camada densa intermediária possuía 1.200 neurônios. Essa configuração resultou em mais de 20 milhões de parâmetros, concentrados principalmente na camada de saída. Como o treinamento foi realizado com apenas 500 reviews, a arquitetura estava superdimensionada para o volume de dados disponível.

O primeiro ajuste arquitetural foi reduzir o vocabulário máximo para 5.000 tokens e a camada densa intermediária para 256 neurônios. Essa alteração diminuiu significativamente o número de parâmetros e reduziu o tempo de treinamento. Entretanto, antes de continuar a otimização da arquitetura, foi necessário revisar a preparação dos dados, pois havia problemas metodológicos capazes de comprometer a avaliação do modelo.

As sequências utilizadas como entrada possuem tamanho máximo igual a 16. Contextos menores precisaram ser completados com padding. Como o valor `0` representa padding, foi adicionado `mask_zero=True` à camada `Embedding`, permitindo que a LSTM ignore essas posições artificiais. Para manter compatibilidade com a execução acelerada por GPU, o padding passou a ser aplicado à direita. A separação entre entrada e alvo também passou a ocorrer antes do padding: o contexto foi preenchido com zeros à direita, enquanto o token seguinte foi armazenado separadamente como valor esperado.

Outro ajuste relevante foi a separação adequada dos conjuntos de treino, validação e teste. Inicialmente, as janelas foram geradas antes da divisão dos dados. Essa abordagem colocou potencialmente em conjuntos diferentes sequências muito semelhantes derivadas da mesma review, produzindo métricas excessivamente otimistas. A divisão passou a ser realizada no nível das reviews completas, antes da vetorização e antes da geração das janelas. Dessa forma, todas as sequências produzidas por uma mesma review permaneceram em um único conjunto.

Foram criados três conjuntos independentes: treino, validação e teste. A distribuição adotada foi de 64% das reviews para treino, 16% para validação e 20% para teste. Como as reviews possuem tamanhos distintos, a proporção de sequências geradas não é exatamente igual à proporção de documentos. Após a divisão, foram obtidas aproximadamente 62% das sequências para treino, 18% para validação e 20% para teste, valores considerados adequados.

O vocabulário passou a ser construído exclusivamente a partir das reviews de treino. Isso evitou que informações dos conjuntos de validação ou teste influenciassem a preparação do modelo. Palavras ausentes do vocabulário foram convertidas para o token `[UNK]`. Essa decisão tornou necessário analisar o papel desse token durante o treinamento.

Foi identificado que `[UNK]` representava 7,25% dos alvos de treino, 14,16% dos alvos de validação e 14,98% dos alvos de teste. Treinar o modelo para prever `[UNK]` não era desejável, pois esse token agrega palavras distintas em uma classe genérica e não pode ser convertido em uma palavra significativa durante a geração textual. Além disso, a função geradora já bloqueava explicitamente a previsão de `[UNK]`. A solução adotada foi manter `[UNK]` nos contextos de entrada, mas remover sequências cujo alvo esperado fosse `[UNK]`.

Em seguida, foi criado um baseline baseado exclusivamente na frequência global dos tokens. Esse baseline ignora completamente o contexto e representa um modelo hipotético que sempre prevê as cinco palavras mais frequentes do corpus de treino: `the`, `and`, `a`, `of` e `to`. Sua acurácia top-5 na validação seria de 19,46%. Esse valor passou a representar a referência mínima que o modelo contextual deveria superar.

A primeira arquitetura avaliada após as correções utilizava `Embedding(100)`, `LSTM(256)`, `Dropout(0.2)`, `Dense(256)` e uma camada de saída com 5.000 classes. Com `learning_rate=1e-4`, o modelo superou o baseline, mas convergiu lentamente. O top-5 de validação começou a ultrapassar o baseline somente após várias épocas, indicando que o modelo inicialmente se comportava de forma semelhante a um preditor baseado em frequência.

Para melhorar a convergência, o `learning_rate` foi aumentado para `3e-4`. Também foram adicionados os callbacks `ReduceLROnPlateau` e `EarlyStopping`. O primeiro reduz automaticamente a taxa de aprendizado quando a `val_loss` deixa de melhorar. O segundo encerra o treinamento após um número definido de épocas sem progresso e restaura os pesos correspondentes à melhor época. Com esse ajuste, o modelo convergiu mais rapidamente e atingiu melhor resultado: `val_loss` próxima de 5,7362 e top-5 de validação próxima de 27,45%.

A geração qualitativa ainda apresentava frases fracas e pouco naturais. Entretanto, o modelo já demonstrava algum aprendizado contextual, pois suas probabilidades deixaram de reproduzir exatamente a frequência global dos tokens. Por exemplo, após o contexto `we liked`, palavras como `the`, `it` e `that` passaram a receber probabilidades mais elevadas, embora palavras inadequadas ainda aparecessem entre as previsões mais prováveis.

O passo seguinte foi remover a camada intermediária `Dense(256)`. Como a saída da LSTM já possui uma representação vetorial capaz de alimentar diretamente a camada `softmax`, a camada densa adicional poderia estar apenas aumentando a capacidade de memorização do modelo. A remoção produziu melhora quantitativa e qualitativa. A menor `val_loss` caiu para aproximadamente 5,6651 e a top-5 de validação passou para aproximadamente 28,03%. A geração também deixou de apresentar algumas repetições triviais observadas anteriormente.

Em seguida, foi avaliada a redução da LSTM de 256 para 128 unidades, mantendo `EMBED_DIM=100` e os demais parâmetros inalterados. Essa alteração reduziu aproximadamente 41% dos parâmetros do modelo. A perda quantitativa foi pequena: a melhor `val_loss` passou de aproximadamente 5,6651 para 5,6776 e a top-5 de validação passou de aproximadamente 28,03% para 27,86%.

Apesar da proximidade das métricas, a análise qualitativa mostrou piora perceptível. A versão com `LSTM(128)` passou a gerar continuações excessivamente genéricas e repetitivas:

```
the movie was → the movie the
we liked → a movie the
the picture was → the movie the
the music was → the movie the
```

A inspeção das probabilidades mostrou que contextos distintos, como `the movie was`, `the picture was` e `the music was`, produziam rankings muito semelhantes. Isso indica que a versão reduzida perdeu parte da capacidade de distinguir informações semânticas relevantes do contexto. Para um modelo cujo objetivo é gerar três palavras coerentes, essa perda qualitativa é importante, mesmo que a diferença na métrica top-5 seja pequena.

Por esse motivo, a LSTM com 256 unidades foi mantida como referência principal. O próximo experimento consistiu em reduzir apenas o tamanho do embedding de 100 para 64 dimensões, mantendo a LSTM com 256 unidades, `Dropout(0.2)`, ausência de camada densa intermediária, `learning_rate=3e-4` e os mesmos callbacks. O objetivo foi verificar se a redução do embedding diminuiria a complexidade do modelo sem prejudicar sua capacidade de gerar três palavras semanticamente relevantes para o contexto apresentado.

A redução do embedding diminuiu o total de parâmetros de aproximadamente 2,15 milhões para 1,93 milhão, uma economia próxima de 10%. Entretanto, essa simplificação prejudicou o desempenho quantitativo. A menor `val_loss` passou de aproximadamente 5,6651 para 5,7152, enquanto a acurácia top-5 de validação na melhor época caiu de aproximadamente 28,03% para 26,79%.

A análise qualitativa também mostrou piora. A versão com `EMBED_DIM=64` passou a gerar praticamente a mesma continuação para diferentes contextos:

```
the movie was → the movie the
we liked → the movie the
the picture was → the movie the
the music was → the movie the
the overall opinion of the movie was → the movie the
```

A inspeção das probabilidades confirmou que contextos semanticamente diferentes produziam distribuições muito semelhantes. Termos genéricos e frequentes, como `the`, `a`, `this`, `i` e `of`, continuavam dominando as primeiras posições. Isso sugere que a redução do embedding diminuiu a capacidade do modelo de preservar diferenças semânticas relevantes entre os tokens.

Dessa forma, a economia de parâmetros obtida com `EMBED_DIM=64` não compensou a perda quantitativa nem a piora na geração das três palavras adicionais. A configuração com `EMBED_DIM=100` e `LSTM(256)` permaneceu como referência para os experimentos seguintes.

Em seguida, o tamanho do embedding foi aumentado de 100 para 128 dimensões, mantendo inalterados os demais componentes da arquitetura. O objetivo foi verificar se uma representação vetorial ligeiramente maior permitiria preservar mais informações semânticas dos tokens sem aumentar excessivamente a complexidade do modelo.

Com `EMBED_DIM=128`, o total de parâmetros passou de aproximadamente 2,15 milhões para 2,32 milhões, um aumento próximo de 7,8%. A menor `val_loss` obtida foi de aproximadamente 5,6579, enquanto a acurácia top-5 de validação na melhor época atingiu aproximadamente 28,08%. O resultado foi ligeiramente superior ao obtido com `EMBED_DIM=100`, cuja menor `val_loss` havia sido de 5,6651 e cuja acurácia top-5 havia atingido 28,03%.

Entretanto, a diferença quantitativa entre as duas configurações foi pequena. A redução da `val_loss` foi de apenas 0,0072 e o ganho na acurácia top-5 foi de aproximadamente 0,05 ponto percentual. Embora a configuração com `EMBED_DIM=128` tenha produzido o melhor resultado quantitativo até aquele momento, a melhora observada não foi suficiente para demonstrar uma vantagem expressiva em relação ao embedding de 100 dimensões.

A análise qualitativa também mostrou ganhos limitados. A geração permaneceu pouco natural e ainda apresentou baixa diferenciação entre contextos distintos:

```
the movie was → the movie of
we liked → the movie to
the picture was → the movie of
the music was → the movie of
the overall opinion of the movie was → the film of
```

Apesar de algumas variações, como a geração de `the film of` em um dos contextos, as continuações ainda eram predominantemente compostas por palavras frequentes e genéricas. A inspeção das probabilidades confirmou que contextos como `the movie was`, `the picture was` e `the music was` continuavam produzindo rankings semelhantes.

Por fim, foi avaliado um aumento mais significativo do embedding, de 128 para 256 dimensões. Os demais parâmetros foram novamente mantidos inalterados: `LSTM(256)`, `Dropout(0.2)`, ausência de camada densa intermediária, `learning_rate=3e-4` e os mesmos callbacks.

Com `EMBED_DIM=256`, o modelo passou a possuir aproximadamente 3,09 milhões de parâmetros, representando um aumento próximo de 43,7% em relação à configuração com `EMBED_DIM=100`. A menor `val_loss` foi reduzida para aproximadamente 5,6152, enquanto a acurácia top-5 de validação na melhor época atingiu aproximadamente 28,42%. Esse foi o melhor resultado quantitativo entre os diferentes tamanhos de embedding avaliados.

A comparação consolidada foi:

`EMBED_DIM`	Parâmetros aproximados	Melhor `val_loss`	Top-5 de validação na melhor época
64	1,93 milhão	5,7152	26,79%
100	2,15 milhões	5,6651	28,03%
128	2,32 milhões	5,6579	28,08%
256	3,09 milhões	5,6152	28,42%

Os resultados sugerem que o aumento da dimensionalidade do embedding contribuiu para melhorar gradualmente a qualidade probabilística do modelo. A redução para 64 dimensões prejudicou claramente o desempenho, enquanto os aumentos para 128 e 256 dimensões produziram ganhos sucessivos na `val_loss` e na acurácia top-5.

Entretanto, a melhora qualitativa continuou limitada. Com `EMBED_DIM=256`, foram geradas as seguintes continuações:

```
the movie was → a movie to
we liked → a movie to
the picture was → the movie of
the music was → a movie and
the overall opinion of the movie was → a movie and
```

A versão com 256 dimensões apresentou maior variação do que as configurações reduzidas e evitou o colapso recorrente em `the movie the`. Ainda assim, as três palavras geradas permaneceram genéricas e pouco adequadas semanticamente aos diferentes contextos apresentados. A inspeção das probabilidades também mostrou que palavras funcionais frequentes, como `a`, `to`, `the`, `this` e `and`, continuavam ocupando as primeiras posições.

Dessa forma, `EMBED_DIM=256` apresentou o melhor desempenho quantitativo entre os valores avaliados e uma melhora qualitativa parcial em relação às versões menores. Contudo, o aumento da dimensionalidade do embedding não foi suficiente, isoladamente, para produzir continuações semanticamente satisfatórias de três palavras.

Os testes realizados até esse momento produziram alguns achados uteis:

Alteração	Resultado
Remover `Dense(256)`	Melhorou
Reduzir `LSTM(256)` para `LSTM(128)`	Preservou métricas, mas piorou geração
Reduzir `EMBED_DIM=100` para `64`	Piorou métricas e geração
Aumentar `EMBED_DIM=100` para `256`	Melhorou ligeiramente métricas e geração

Portanto, acreditamos que novos cortes de arquitetura provavelmente não ajudarão. O principal limitador restante parece ser a pequena quantidade de textos de treino: apenas 320 reviews. Para obter melhora significativa na geração das três palavras, o próximo teste mais informativo foi manter a melhor arquitetura e aumentar a quantidade de reviews utilizadas no treinamento.

Até então, as 500 reviews selecionadas inicialmente haviam sido divididas entre treino, validação e teste. Essa estratégia reservava apenas 64% dos textos para o treinamento efetivo. Como o dataset IMDB continha reviews adicionais ainda não utilizadas, a divisão foi reorganizada. As 500 reviews originais passaram a ser utilizadas exclusivamente para desenvolvimento do modelo: 400 foram destinadas ao treinamento e 100 à validação. Um novo conjunto externo de teste, contendo outras 500 reviews, foi separado a partir dos textos que não faziam parte da amostra original. Essa reorganização preservou a independência do conjunto de teste e, ao mesmo tempo, aumentou a quantidade de textos disponíveis para ajuste dos pesos. O vocabulário continuou sendo construído exclusivamente com as reviews do conjunto de treino. O conjunto externo de teste permaneceu isolado durante a escolha da arquitetura e dos hiperparâmetros.

Com a nova divisão, o conjunto de treino passou a gerar 332 batches por época, em comparação com os 259 batches observados anteriormente. O treinamento permaneceu estável e o melhor resultado segundo a `val_loss` ocorreu na época 21:

```
loss:               5,2078
top_5_accuracy:     30,89%
val_loss:           5,6485
val_top_5_accuracy: 28,73%
```

Como o conjunto de treino e validação havia sido alterado, o baseline também foi recalculado. As cinco palavras mais frequentes permaneceram:

```
the
and
a
of
to
```

A acurácia top-5 do baseline foi de 19,33%, valor muito próximo dos 19,46% observados anteriormente. O modelo contextual atingiu 28,73% de acurácia top-5 na melhor época, superando o baseline em 9,40 pontos percentuais.

A comparação com o treinamento anterior foi:

Configuração	Baseline top-5	Top-5 do modelo na melhor `val_loss`	Ganho absoluto
320 reviews de treino	19,46%	28,42%	8,96 p.p.
400 reviews de treino	19,33%	28,73%	9,40 p.p.

A melhora quantitativa foi modesta, mas consistente. O aumento de 320 para 400 reviews permitiu elevar ligeiramente o ganho do modelo sobre o baseline de frequência.

Entretanto, a análise qualitativa ainda mostrou limitações importantes. As três palavras geradas permaneceram genéricas e pouco sensíveis ao contexto:

```
the movie was → a movie to
we liked → the movie the
the picture was → a movie a
the music was → a movie to
the overall opinion of the movie was → a movie of
```

A inspeção das probabilidades também revelou que contextos terminados em `was` continuavam produzindo distribuições semelhantes. Termos como `a`, `to`, `the`, `in` e `this` permaneciam entre as previsões mais prováveis, enquanto palavras semanticamente mais específicas ainda apareciam com baixa frequência.

Dessa forma, o aumento do conjunto de treino de 320 para 400 reviews produziu uma melhora quantitativa limitada, mas não resolveu o problema qualitativo. O modelo continuou aprendendo relações contextuais reais, pois manteve vantagem clara sobre o baseline, mas ainda não conseguiu gerar três palavras semanticamente satisfatórias para os contextos apresentados.

O passo seguinte foi aumentar a quantidade de dados de treinamento, já que modificações no modelo não pareciam apresentar ganhos significativos na avaliação qualitativa que realizamos a partir das frases geradas. Dessa forma passamos para 3000 reviews nosso conjunto de treino, mantendo os 500 reviews de teste a parte. Com o aumento para 3000 reviews de treino, o objetivo foi verificar se a limitação qualitativa observada até então estava mais relacionada à baixa quantidade de dados do que à arquitetura em si. A arquitetura foi mantida simples, com uma camada de entrada, uma camada de `Embedding`, uma camada `LSTM`, uma camada `Dropout` e uma camada de saída `Dense` com ativação `softmax`.

Pudemos observar um ganho significativo nas métricas, também observamos uma pequena melhora na avaliação qualitativa. Entretanto, nos pareceu que chegavamos num limitanto com a simplicidade e a quantidade de informação de treino disponível. Chegamos a iniciar um treino com 5.000 reviews mas rapidamente ficou claro que o poder de processamento disponível tornaria inviável o tempo de treinamento.

Nesta etapa, pensamos em avaliar técnicas alternativas, como por exemplo transfer learning. Por conta disso foi introduzido o uso de embeddings pré-treinados GloVe. A camada `Embedding` passou a ser inicializada com a matriz de embeddings construída a partir dos vetores GloVe, mantendo `mask_zero=True` para ignorar tokens de padding. No modelo principal, os embeddings foram mantidos congelados com `trainable=False`, preservando as relações semânticas aprendidas no pré-treinamento. A arquitetura utilizada foi composta por `Embedding` com pesos GloVe, `LSTM(256)`, `Dropout(0.5)` e camada de saída `Dense(VOCAB_SIZE, activation="softmax")`.

O treinamento foi realizado com `learning_rate=3e-4`, função de perda `sparse_categorical_crossentropy` e métrica `SparseTopKCategoricalAccuracy(k=5)`, nomeada como `top_5_accuracy`. Os callbacks `EarlyStopping` e `ReduceLROnPlateau` continuaram monitorando `val_loss`, pois a perda de validação foi adotada como critério principal de seleção do modelo. Dessa forma, a melhor época passou a ser definida pela menor `val_loss`, e não pela maior top-5 accuracy.

O modelo treinado com GloVe congelado apresentou melhora importante em relação aos experimentos anteriores com menor quantidade de dados. A menor `val_loss` observada foi aproximadamente 5,0909, com top-5 de validação em torno de 35,36%. Esse resultado representou um salto relevante em relação aos modelos anteriores, que estavam na faixa de aproximadamente 28% de top-5 accuracy na validação. A melhora indica que o aumento do volume de dados, combinado com o uso de embeddings pré-treinados, contribuiu mais para o desempenho do que as pequenas alterações arquiteturais avaliadas anteriormente.

A geração gulosa com cinco tokens adicionais mostrou que o modelo passou a produzir continuações mais estruturadas, embora ainda genéricas. Alguns exemplos foram:

```
My opinion about the movie was that it was
→ My opinion about the movie was that it was a good movie i have

The actors in this move did a
→ The actors in this move did a great film is a good

The best of music and the soundtrack was particularly
→ The best of music and the soundtrack was particularly written and the actors are

what i liked the most in the movie
→ what i liked the most in the movie was a good movie i

The ending of the film reserved a
→ The ending of the film reserved a few years is a young
```

A análise qualitativa mostrou melhora parcial. Em alguns casos, como `it was a good movie`, o modelo produziu uma continuação plausível. Entretanto, em outros exemplos, as frases ainda apresentaram problemas sintáticos e semânticos, como `did a great film is a good` e `reserved a few years is a young`. Portanto, mesmo com ganho quantitativo expressivo, a geração gulosa ainda permaneceu limitada, especialmente quando avaliada por coerência textual.

Em seguida, foi realizado um grid search para avaliar variações da arquitetura. Os principais fatores avaliados foram o tamanho da camada recorrente e a possibilidade de liberar ou não o treinamento dos embeddings GloVe. Essa etapa gerou quatro configurações principais. O Trial 0000 reproduziu a configuração com GloVe congelado e `LSTM(256)`, funcionando como uma segunda execução da configuração base. O resultado foi muito próximo ao modelo geral, com `val_loss` aproximadamente 5,1117, confirmando que pequenas variações entre execuções podem ocorrer por inicialização aleatória, dropout e ordem dos batches.

O Trial 0001 manteve os embeddings GloVe congelados, mas utilizou uma camada recorrente maior. Esse teste foi importante para verificar se o aumento da capacidade da LSTM, isoladamente, seria suficiente para melhorar a generalização sem alterar os embeddings pré-treinados. Embora a maior capacidade pudesse aumentar o poder de representação do modelo, ela também aumentava o risco de overfitting. A análise das curvas indicou que o aumento da camada recorrente não trouxe ganho suficientemente forte para justificar a escolha dessa configuração em relação ao modelo base.

Os Trials 0002 e 0003 apresentaram os melhores resultados quantitativos do grid search. A diferença mais importante desses trials foi a liberação do treinamento dos embeddings GloVe. O melhor resultado quantitativo foi observado no Trial 0003, com top-5 de validação próxima de 37,27%, enquanto o Trial 0002 ficou muito próximo, com top-5 de validação próxima de 37,12%. Esses valores superaram o modelo geral, que ficou em torno de 35,36%.

Entretanto, essa melhora quantitativa precisou ser interpretada com cautela. A liberação do treinamento dos embeddings GloVe altera uma premissa importante do uso de transfer learning. Como os vetores GloVe já carregam relações semânticas aprendidas em um corpus grande, seu ajuste fino deve ser feito com cuidado, preferencialmente com uma taxa de aprendizado menor para a camada de embedding. No experimento realizado, os embeddings foram treinados com a mesma taxa de aprendizado das demais camadas, sem um controle específico para preservar a estrutura semântica original.

Esse ponto introduziu um risco metodológico. A melhora na métrica pode ter ocorrido porque os embeddings se ajustaram ao conjunto de treino, mas sem necessariamente produzir uma melhoria qualitativa real na geração textual. Esse risco ficou mais evidente quando a geração gulosa do melhor modelo do grid search foi comparada com a geração do modelo geral usando os mesmos parâmetros. Para os exemplos avaliados, as frases geradas foram idênticas:

```
My opinion about the movie was that it was
→ My opinion about the movie was that it was a good movie i have

The actors in this move did a
→ The actors in this move did a great film is a good

The best of music and the soundtrack was particularly
→ The best of music and the soundtrack was particularly written and the actors are

what i liked the most in the movie
→ what i liked the most in the movie was a good movie i

The ending of the film reserved a
→ The ending of the film reserved a few years is a young
```

Isso mostrou que, sob geração gulosa, o ganho de top-5 accuracy dos melhores trials do grid search não se traduziu em melhoria perceptível na qualidade das frases geradas. Como a geração gulosa seleciona sempre o token de maior probabilidade, é possível que o modelo do grid search tenha melhorado a posição do token correto dentro dos cinco mais prováveis, sem alterar o token mais provável escolhido em cada passo da geração.

Dessa forma, o Trial 0003 foi considerado o melhor modelo quantitativo do grid search, mas não necessariamente a escolha mais segura para o modelo final. Como sua vantagem dependia da liberação do treinamento do GloVe sem taxa de aprendizado diferenciada, e como a avaliação qualitativa não mostrou melhora em relação ao modelo geral, optamos por uma escolha mais conservadora: manter o modelo geral com embeddings GloVe congelados.

A escolha final, portanto, priorizou a segurança metodológica. O modelo selecionado preserva os embeddings pré-treinados, utiliza uma arquitetura simples e apresentou desempenho quantitativo competitivo, além de comportamento qualitativo equivalente ao melhor modelo do grid search nos exemplos analisados. A arquitetura final escolhida foi:

```
Input(MAX_LEN)
→ Embedding(VOCAB_SIZE, EMBED_DIM, weights=GloVe, mask_zero=True, trainable=False)
→ LSTM(256)
→ Dropout(0.5)
→ Dense(VOCAB_SIZE, softmax)
```

No treinamento final, foram combinados os conjuntos de treino e validação utilizados durante o desenvolvimento, mantendo o conjunto de teste externo completamente isolado. Como a validação já havia sido utilizada para escolha da arquitetura, dos hiperparâmetros e da época de referência, o modelo final foi treinado sem conjunto de validação, utilizando 29 épocas, valor associado à melhor época observada anteriormente para a configuração escolhida.

A curva do treinamento final mostrou comportamento esperado: a `loss` de treino caiu de forma contínua ao longo das épocas, enquanto a `top_5_accuracy` de treino aumentou progressivamente. Ao final do treinamento, a `loss` de treino atingiu aproximadamente 4,8659. Como não havia mais conjunto de validação nessa etapa, o gráfico final contém apenas as métricas de treino. A avaliação real de generalização foi reservada para o conjunto de teste externo.

No conjunto de teste, o modelo final obteve os seguintes resultados:

```
loss:           5,060905
top_5_accuracy: 35,3125%
```

Esse resultado ficou próximo ao desempenho observado durante a validação da configuração escolhida, indicando que o modelo manteve comportamento consistente em dados não utilizados durante a seleção da arquitetura. A top-5 accuracy final de aproximadamente 35,31% também permaneceu claramente acima do baseline baseado em frequência, confirmando que o modelo aprendeu padrões contextuais além da simples escolha das palavras mais frequentes do corpus.

A geração gulosa no modelo final apresentou resultados melhores do que os observados nas primeiras etapas do experimento, embora ainda com limitações sintáticas e semânticas. Foram obtidas as seguintes continuações:

```
My opinion about the movie was that it was
→ My opinion about the movie was that it was a good movie

The actors in this move did a
→ The actors in this move did a lot of the

The best of music and the soundtrack was particularly
→ The best of music and the soundtrack was particularly the acting was

what i liked the most in the movie
→ what i liked the most in the movie i have seen

The ending of the film reserved a
→ The ending of the film reserved a young woman who
```

A primeira continuação, `it was a good movie`, foi simples, mas coerente com o contexto. A última, `a young woman who`, também apresentou uma sequência gramaticalmente plausível, ainda que não necessariamente adequada ao início da frase. Por outro lado, exemplos como `did a lot of the` e `was particularly the acting was` mostram que o modelo ainda possui dificuldade em preservar coerência sintática por múltiplos tokens quando a geração é feita de forma gulosa.

Também foi avaliada a geração probabilística, utilizando amostragem para selecionar os próximos tokens em vez de sempre escolher a palavra mais provável. Essa abordagem produziu maior diversidade nas frases geradas. Em alguns casos, surgiram continuações plausíveis, como:

```
My opinion about the movie was that it was a good movie i have
My opinion about the movie was that it was a bad movie is not
The ending of the film reserved a woman who is an adult
The ending of the film reserved a man and has to be
```

Entretanto, a geração probabilística também gerou sequências instáveis ou repetitivas, como:

```
what i liked the most in the movie that i am not not
what i liked the most in the movie i have seen i am
The actors in this move did a movie is so good it
```

Esses resultados mostram que a amostragem probabilística aumenta a variedade das continuações, mas não resolve completamente os problemas de coerência. O modelo aprendeu padrões linguísticos frequentes do domínio de reviews de filmes, como `a good movie`, `a bad movie`, `i have seen` e `the acting was`, mas ainda tem dificuldade em construir frases longas e semanticamente bem ajustadas ao contexto inicial.

A comparação entre geração gulosa e geração probabilística indica que a geração gulosa é mais estável, porém tende a produzir continuações genéricas. A geração probabilística, por sua vez, explora alternativas mais variadas, mas aumenta o risco de sequências gramaticalmente ruins ou semanticamente inconsistentes. Para o objetivo do exercício, que exigia gerar ao menos três palavras adicionais com algum significado relevante para o contexto, o modelo final atende parcialmente ao requisito: algumas continuações são plausíveis e relacionadas ao domínio, mas a qualidade ainda é irregular.

De forma geral, o experimento mostrou que os principais ganhos vieram da correção metodológica da preparação dos dados, do aumento da quantidade de reviews utilizadas no treinamento e do uso de embeddings GloVe pré-treinados. Alterações menores na arquitetura, como redução da LSTM, redução do embedding ou adição de camadas densas intermediárias, não produziram melhora consistente. A escolha final por uma arquitetura simples com GloVe congelado, `LSTM(256)`, `Dropout(0.5)` e saída `softmax` mostrou-se a alternativa mais segura e metodologicamente consistente entre as opções avaliadas.